# IV Convolutional Neural Networks

# Crash Course in Convolutional Neural Networks

23

Convolutional neural networks are a powerful artificial neural network technique. These networks preserve the spatial structure of the problem and were developed for object recognition tasks such as handwritten digit recognition. They are popular because people can achieve state-of-the-art results on challenging computer vision and natural language processing tasks. In this chapter, you will discover convolutional neural networks for deep learning, also called ConvNets or CNNs. After completing this crash course, you will know:

- ▷ The building blocks used in CNNs, such as convolutional layers and pool layers
- ▷ How the building blocks fit together with a short worked example
- ▷ Best practices for configuring CNNs on your object recognition tasks
- ▷ References for state-of-the-art networks applied to complex machine learning problems

Let's get started.

## Overview

This chapter is divded into four sections; they are:

- ▷ The Case for Convolutional Neural Networks
- ▷ Building Blocks of Convolutional Neural Networks
- ▷ Convolutional Layers
- ▷ Pooling Layers
- ▷ Fully Connected Layers
- ▷ Worked Example of a Convolutional Neural Network
- ▷ Convolutional Neural Networks Best Practices

## 23.1 The Case for Convolutional Neural Networks

Given a dataset of grayscale images with the standardized size of $32 \times 32$ pixels each, a traditional feedforward neural network would require 1024 input weights (plus one bias). This

is fair enough, but the flattening of the image matrix of pixels to a long vector of pixel values loses all the spatial structure in the image. Unless all the images are perfectly resized, the neural network will have great difficulty with the problem.

Convolutional neural networks use fewer weights. They also expect and preserve the spatial relationship between pixels by learning internal feature representations using small squares of input data. Features are learned and used across the whole image, allowing the objects in the images to be shifted or translated in the scene but still detectable by the network. This is why the network is so useful for object recognition in photographs, picking out digits, faces, objects, and so on with varying orientations. In summary, below are some of the benefits of using convolutional neural networks:

- ▷ They use fewer parameters (weights) to learn than a fully connected network
- ▷ They are designed to be invariant to object position and distortion in the scene
- ▷ They automatically learn and generalize features from the input domain

## 23.2 Building Blocks of Convolutional Neural Networks

There are three types of layers in a convolutional neural network:

1. Convolutional Layers
2. Pooling Layers
3. Fully-Connected Layers

## 23.3 Convolutional Layers

Convolutional layers are comprised of filters and feature maps.

### Filters

Filters are essentially the *neurons* of the layer. They take weighted inputs and output a value. The input size is a fixed square called a patch or a *receptive field*. If the convolutional layer is an input layer, the input patch will be the pixel values. If deeper in the network architecture, the convolutional layer will take input from a feature map from the previous layer.

### Feature Maps

A feature map is the output of one filter applied to the previous layer. A given filter is drawn across the entire previous layer and moved one pixel at a time. Each position results in the activation of the neuron, and the output is collected in the feature map. You can see that if the receptive field is moved one pixel from activation to activation, then the field will overlap with the previous activation by (field width$-1$) input values.

The distance that filter is moved across the input from the previous layer for each activation is referred to as the stride. If the size of the previous layer is not cleanly divisible by the size of the filter's receptive field and the size of the stride, then it is possible for the receptive field

to attempt to read off the edge of the input feature map. In this case, techniques like zero padding can be used to invent mock inputs with zero values for the receptive field to read.

## 23.4 Pooling Layers

The pooling layers downsample the previous layer's feature map. Pooling layers follow a sequence of one or more convolutional layers and are intended to consolidate the features learned and expressed in the previous layer's feature map. As such, pooling may be considered a technique to compress or generalize feature representations and generally reduce the overfitting of the training data by the model.

They, too, have a receptive field, often much smaller than the convolutional layer. Also, the stride or number of inputs that the receptive field is moved for each activation is often equal to the size of the receptive field to avoid any overlap. Pooling layers are often very simple, taking the average (average pooling) or the maximum (max pooling) of the input values in order to create its own feature map.

## 23.5 Fully Connected Layers

Fully connected layers are the normal flat feedforward neural network layer. These layers may have a nonlinear activation function or a softmax activation in order to output probabilities of class predictions. Fully connected layers are used at the end of the network after feature extraction and consolidation have been performed by the convolutional and pooling layers. They are used to create final nonlinear combinations of features and for making predictions by the network.

## 23.6 Worked Example of a Convolutional Neural Network

You now know about convolutional, pooling, and fully connected layers. Let's make this more concrete by working through how these three layers may be connected together.

### Image Input Data

Let's assume you have a dataset of grayscale images. Each image has the same size of 32 pixels wide and 32 pixels high, and pixel values are between 0 and 255, e.g., a matrix of $32 \times 32 \times 1$ or 1024 pixel values. Image input data is expressed as a 3-dimensional matrix of width $\times$ height $\times$ channels. If you were using color images in the example, you would have three channels for the red, green, and blue pixel values, e.g., $32 \times 32 \times 3$.

### Convolutional Layer

Define a convolutional layer with ten filters, and a receptive field 5 pixels wide and 5 pixels high, and a stride length of 1. Because each filter can only get input from (i.e., *see*) $5 \times 5$ or 25 pixels at a time, you can calculate that each will require $25 + 1$ input weights (plus 1 for the bias input). Dragging the $5 \times 5$ receptive field across the input image data with a stride

width of 1 will result in a feature map of $28 \times 28$ output values or 784 distinct activations per image.

You have ten filters, so ten different $28 \times 28$ feature maps or 7,840 outputs will be created for one image. Finally, you know you have 26 inputs per filter, ten filters, and $28 \times 28$ output values to calculate per filter. Therefore, you have a total of $26 \times 10 \times 28 \times 28$ or 203,840 *connections* in your convolutional layer if you want to phrase it using traditional neural network nomenclature. Convolutional layers also make use of a nonlinear transfer function as part of the activation, and the rectifier activation function is the popular default to use.

### Pooling Layer

You can define a pooling layer with a receptive field with a width of 2 inputs and a height of 2 inputs. You can also use a stride of 2 to ensure that there is no overlap. This results in feature maps that are one-half the size of the input feature maps, from ten different $28 \times 28$ feature maps as input to ten different $14 \times 14$ feature maps as output. You will use a `max()` operation for each receptive field so that the activation is the maximum input value.

### Fully Connected Layer

Finally, you can flatten out the square feature maps into a traditional flat, fully connected layer. You can define the fully connected layer with 200 hidden neurons, each with $10 \times 14 \times 14$ input connections, or $1,960 + 1$ weights per neuron. That is a total of 392,200 connections and weights to learn in this layer. You can use a sigmoid or softmax transfer function to output probabilities of class values directly.

## 23.7 Convolutional Neural Networks Best Practices

Now that you know about the building blocks for a convolutional neural network and how the layers hang together, you can review some best practices to consider when applying them.

- ▷ **Input Receptive Field Dimensions**: The default is 2D for images but could be 1D for words in a sentence or 3D for a video that adds a time dimension.
- ▷ **Receptive Field Size**: The patch should be as small as possible but large enough to *see* features in the input data. It is common to use $3 \times 3$ on small images and $5 \times 5$ or $7 \times 7$ and more on larger image sizes.
- ▷ **Stride Width**: Use the default stride of 1. It is easy to understand, and you don't need padding to handle the receptive field falling off the edge of your images. This could be increased to 2 or larger for larger images.
- ▷ **Number of Filters**: Filters are the feature detectors. Generally, fewer filters are used at the input layer, and increasingly more filters are used at deeper layers.
- ▷ **Padding**: Set to zero and called zero padding when reading non-input data. This is useful when you cannot or do not want to standardize input image sizes or when you want to use receptive field and stride sizes that do not neatly divide up the input image size.

- ▷ **Pooling**: Pooling is a destructive or generalization process to reduce overfitting. The receptive field size is almost always set to $2 \times 2$ with a stride of 2 to discard 75% of the activations from the output of the previous layer.
- ▷ **Data Preparation**: Consider standardizing input data, both the dimensions of the images and pixel values.
- ▷ **Pattern Architecture**: It is common to pattern the layers in your network architecture. This might be one, two, or some number of convolutional layers followed by a pooling layer. This structure can then be repeated one or more times. Finally, fully connected layers are often only used at the output end and may be stacked one, two, or more deep.
- ▷ **Dropout**: CNNs have a habit of overfitting, even with pooling layers. Dropout should be used, such as between fully connected layers and perhaps after pooling layers.

## 23.8 Further Readings

You have only scratched the surface on convolutional neural networks. The field is moving very fast, and new and interesting architectures and techniques are being discussed and used all the time.

If you are looking for a deeper understanding of the technique, take a look at LeCun, Bottou, et al.'s seminal paper titled "Gradient-Based Learning Applied to Document Recognition". In it, they introduce LeNet applied to handwritten digit recognition and carefully explain the layers and how the network is connected.

There are a lot of tutorials and discussions of CNNs around the web. A few chosen examples are listed below. Personally, I find the explanatory pictures in the posts useful only after understanding how the network hangs together. Many of the explanations are confusing and I recommend you to read LeCun's paper if in doubt.

### Articles

Yann LeCun, Leon Bottou, et al. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE*. Nov. 1998.
http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf

Yann LeCun. *LeNet-5 convolutional neural networks*.
http://yann.lecun.com/exdb/lenet/

*Convolutional Neural Networks*. Stanford Course CS231n. Convolutional Neural Networks for Visual Recognition.
https://cs231n.github.io/convolutional-networks/

Yann LeCun, Koray Kavukcuoglu, and Clement Farabet. "Convolutional Networks and Applications in Vision". In: *Proceedings of IEEE International Symposium on Circuits and Systems*. Paris, France, 2010.
http://yann.lecun.com/exdb/publis/pdf/lecun-iscas-10.pdf

Michael Nielsen. Chapter 6, "Deep Learning". In: *Neural Networks abd Deep Learning*. Determination Press, 2015.
http://neuralnetworksanddeeplearning.com/chap6.html

Andrea Vedaldi and Andrew Zisserman. *VGG Convolutional Neural Networks Practical*. Oxford Visual Geometry Group, 2017.
http://www.robots.ox.ac.uk/~vgg/practicals/cnn/

Denny Britz. *Understanding Convolutional Neural Networks for NLP*. Nov. 2015.
https://www.kdnuggets.com/2015/11/understanding-convolutional-neural-networks-nlp.html

## 23.9 Summary

In this chapter, you discovered convolutional neural networks. You learned about:

- ▷ Why CNNs are needed to preserve spatial structure in your input data and the benefits they provide
- ▷ The building blocks of CNN include convolutional, pooling, and fully connected layers
- ▷ How the layers in a CNN hang together
- ▷ Best practices when applying a CNN to your own problems

In the next chapter, you will look closer to the hyperparameters of a convolutional layer.

# Understanding the Design of a Convolutional Neural Network

24

Convolutional neural networks have been found successful in computer vision applications. Various network architectures are proposed and they are neither magical nor hard to understand. In this chapter, you will make sense of the operation of convolutional layers and their role in a larger convolutional neural network. After finishing this chapter, you will learn:

- How convolutional layers extract features from image
- How different convolutional layers can stack up to build a neural network

Let's get started.

## Overview

This chapter is divided into three sections; they are:

- An Example Network
- Showing the Feature Maps
- Effect of the Convolutional Layers

## 24.1 An Example Network

The following is a program to do image classification on the CIFAR-10 dataset:

```
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dropout, MaxPooling2D, Flatten, Dense
from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.datasets.cifar10 import load_data


(X_train, y_train), (X_test, y_test) = load_data()
```

```
# rescale image
X_train_scaled = X_train / 255.0
X_test_scaled = X_test / 255.0

model = Sequential([
    Conv2D(32, (3,3), input_shape=(32, 32, 3), padding="same",
           activation="relu", kernel_constraint=MaxNorm(3)),
    Dropout(0.3),
    Conv2D(32, (3,3), padding="same",
           activation="relu", kernel_constraint=MaxNorm(3)),
    MaxPooling2D(),
    Flatten(),
    Dense(512, activation="relu", kernel_constraint=MaxNorm(3)),
    Dropout(0.5),
    Dense(10, activation="sigmoid")
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics="sparse_categorical_accuracy")

model.fit(X_train_scaled, y_train, epochs=25, batch_size=32,
          validation_data=(X_test_scaled, y_test))
```

*Listing 24.1: Image classification on CIFAR-10 dataset*

This network should be able to achieve around 70% accuracy in classification. The images are in $32 \times 32$ pixels in RGB color. They are in 10 different classes, and the labels are integers from 0 to 9.

You can print the network using Keras' `summary()` function:

```
...
model.summary()
```

*Listing 24.2: Print a network model*

In this network, the following will be shown on the screen:

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 32, 32, 32)        896

 dropout (Dropout)           (None, 32, 32, 32)        0

 conv2d_1 (Conv2D)           (None, 32, 32, 32)        9248

 max_pooling2d (MaxPooling2D)  (None, 16, 16, 32)      0

 flatten (Flatten)           (None, 8192)              0

 dense (Dense)               (None, 512)               4194816
```

```
 dropout_1 (Dropout)           (None, 512)                0

 dense_1 (Dense)               (None, 10)                 5130

=================================================================
Total params: 4,210,090
Trainable params: 4,210,090
Non-trainable params: 0
_________________________________________________________________
```

*Output 24.1: The network model we created in Listing 24.1*

It is typical in a network for image classification to be comprised of convolutional layers at an early stage, with dropout and pooling layers interleaved. Then, at a later stage, the output from convolutional layers is flattened and processed by some fully connected layers.

## 24.2 Showing the Feature Maps

In the above network, there are two convolutional layers (`Conv2D`). The first layer is defined as follows:

```
Conv2D(32, (3,3), input_shape=(32, 32, 3), padding="same", activation="relu",
       kernel_constraint=MaxNorm(3))
```

*Listing 24.3: The first layer in our network model*

This means the convolutional layer will have a $3 \times 3$ kernel and apply on an input image of $32 \times 32$ pixels and three channels (the RGB colors). therefore, the output of this layer will be 32 channels.

In order to make sense of the convolutional layer, you can check out its kernel. The variable `model` holds the network, and you can find the kernel of the first convolutional layer with the following:

```
...
print(model.layers[0].kernel)
```

*Listing 24.4: Show the kernel of the first layer in our model*

This prints:

```
<tf.Variable 'conv2d/kernel:0' shape=(3, 3, 3, 32) dtype=float32, numpy=
array([[[[-2.30068922e-01,  1.41024575e-01, -1.93124503e-01,
          -2.03153938e-01,  7.71819279e-02,  4.81446862e-01,
          -1.11971676e-01, -1.75487325e-01, -4.01797555e-02,
          ...
           4.64215249e-01,  4.10646647e-02,  4.99733612e-02,
          -5.22711873e-02, -9.20209661e-03, -1.16479330e-01,
           9.25614685e-02, -4.43541892e-02]]]], dtype=float32)>
```

*Output 24.2: The kernel in the first layer of our model*

You can tell that `model.layers[0]` is the correct layer by comparing the name `conv2d` from the above output to the output of `model.summary()`. This layer has a kernel of shape `(3, 3, 3, 32)`, which are the height, width, input channels, and output feature maps, respectively.

Assume the kernel is a NumPy array `k`. A convolutional layer will take its kernel `k[:, :, 0, n]` (a $3 \times 3$ array) and apply on the first channel of the image. Then apply `k[:, :, 1, n]` on the second channel of the image, and so on. Afterward, the result of the convolution on all the channels is added up to become the feature map `n` of the output, where `n`, in this case, will run from 0 to 31 for the 32 output feature maps.

In Keras, you can extract the output of each layer using an extractor model. In the following, you will create a batch with one input image and send it to the network. Then look at the feature maps of the first convolutional layer:

```
...
# Extract output from each layer
extractor = tf.keras.Model(inputs=model.inputs,
                           outputs=[layer.output for layer in model.layers])
features = extractor(np.expand_dims(X_train_scaled[7], 0))

# Show the 32 feature maps from the first layer
l0_features = features[0].numpy()[0]

fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(l0_features[..., i])

plt.show()
```

*Listing 24.5: Showing the feature map from the first convolutional layer*

The above code will print the feature maps like the following:

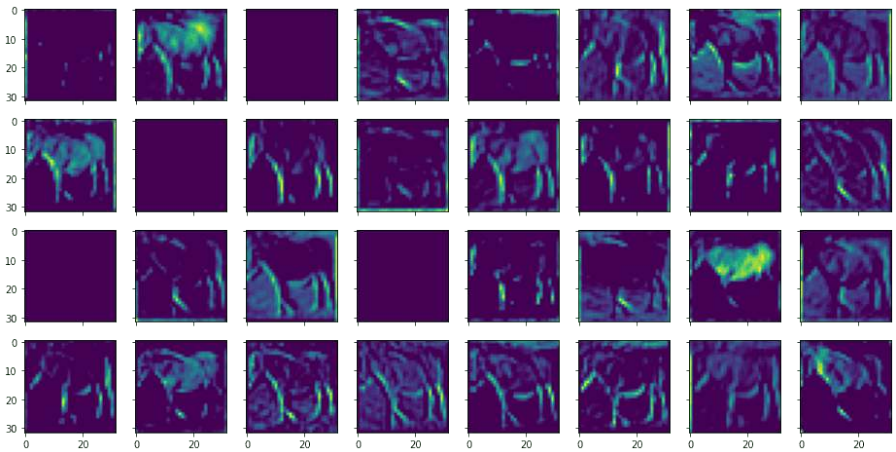

*Figure 24.1: The feature map of the first layer*

This corresponds to the following input image:

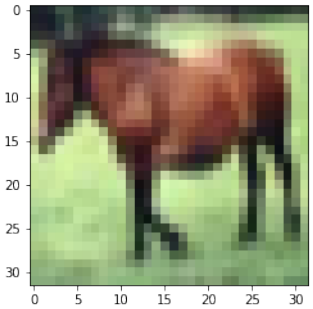

*Figure 24.2: The input image to our network*

You can see that they are called feature maps because they are highlighting certain features from the input image. A feature is identified using a small window (in this case, over a $3 \times 3$ pixels filter). The input image has three color channels. Each channel has a different filter applied, and their results are combined for an output feature.

You can similarly display the feature map from the output of the second convolutional layer as follows:

```
...
# Show the 32 feature maps from the third layer
l2_features = features[2].numpy()[0]

fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(l2_features[..., i])

plt.show()
```

*Listing 24.6: Showing the feature map from the second convolutional layer*

This shows the following:

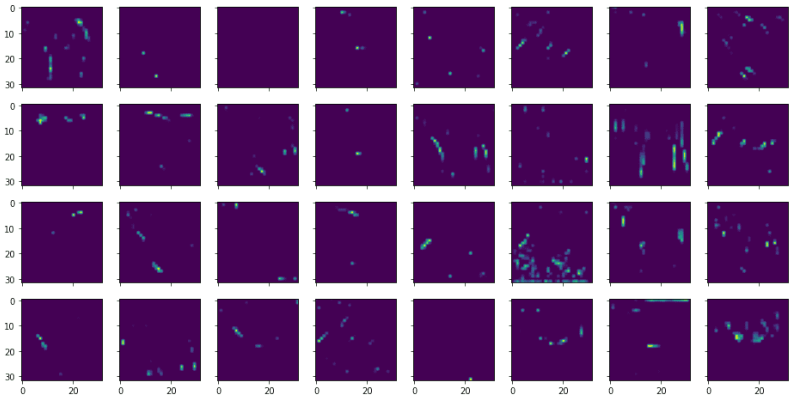

*Figure 24.3: The feature map of the second layer*

From the above, you can see that the features extracted are more abstract and less recognizable.

The following is the complete code to create, train, and visualize the feature maps from a trained network.

```
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dropout, MaxPooling2D, Flatten, Dense
from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.datasets.cifar10 import load_data


(X_train, y_train), (X_test, y_test) = load_data()

# rescale image
X_train_scaled = X_train / 255.0
X_test_scaled = X_test / 255.0

model = Sequential([
    Conv2D(32, (3,3), input_shape=(32, 32, 3), padding="same",
           activation="relu", kernel_constraint=MaxNorm(3)),
    Dropout(0.3),
    Conv2D(32, (3,3), padding="same",
           activation="relu", kernel_constraint=MaxNorm(3)),
    MaxPooling2D(),
    Flatten(),
    Dense(512, activation="relu", kernel_constraint=MaxNorm(3)),
    Dropout(0.5),
    Dense(10, activation="sigmoid")
])

# Train the network with CIFAR10 dataset
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics="sparse_categorical_accuracy")

model.fit(X_train_scaled, y_train, epochs=25, batch_size=32,
          validation_data=(X_test_scaled, y_test))

# Visualize the input image
plt.imshow(X_train_scaled[7])
plt.show()

# Extract output from each layer
extractor = tf.keras.Model(inputs=model.inputs,
                           outputs=[layer.output for layer in model.layers])
features = extractor(np.expand_dims(X_train_scaled[7], 0))

# Show the 32 feature maps from the first layer
l0_features = features[0].numpy()[0]

fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(l0_features[..., i])

plt.show()
```

```
# Show the 32 feature maps from the third layer
l2_features = features[2].numpy()[0]

fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(l2_features[..., i])

plt.show()
```

*Listing 24.7: Visualizing the feature map from a network*

## 24.3 Effect of the Convolutional Layers

The most important hyperparameter to a convolutional layer is the size of the filter. Usually, it is in a square shape, and you can consider that as a *window* or *receptive field* to look at the input image. Therefore, the higher resolution of the image, then you can expect a larger filter.

On the other hand, a filter too large will blur the detailed features because all pixels from the receptive field through the filter will be combined into one pixel at the output feature map. Therefore, there is a trade-off for the appropriate size of the filter.

Stacking two convolutional layers (without any other layers in between) is equivalent to a single convolutional layer with a larger filter. But a typical design to use nowadays is two layers with small filters stacked together rather than one larger with a larger filter, as there are fewer parameters to train.

The exception would be a convolutional layer with a $1 \times 1$ filter. This is usually found as the beginning layer of a network. The purpose of such a convolutional layer is to combine the input channels into one rather than transforming the pixels. Conceptually, this can convert a color image into grayscale, but usually, you can use multiple ways of conversion to create more input channels than merely RGB for the network.

Also, note that in the above network, you are using `Conv2D` for a 2D filter. There is also a `Conv3D` layer for a 3D filter. The difference is whether you apply the filter separately for each channel or feature map or consider the input feature maps stacked up as a 3D array and apply a single filter to transform it altogether. Usually, the former is used as it is more reasonable to consider no particular order in which the feature maps should be stacked.

Besides the size of the receptive field, *padding* and *stride* are two other parameters of a convolutional layer. Padding describes how far the receptive field can go outside the border of the input image. Stride describes how big a step the receptive field would move on the input image to produce the layer's next output.

## 24.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

### Articles

*Convolutional Neural Networks*. Stanford Course CS231n. Convolutional Neural Networks for Visual Recognition.
https://cs231n.github.io/convolutional-networks/

Yann LeCun, Koray Kavukcuoglu, and Clement Farabet. "Convolutional Networks and Applications in Vision". In: *Proceedings of IEEE International Symposium on Circuits and Systems*. Paris, France, 2010.
http://yann.lecun.com/exdb/publis/pdf/lecun-iscas-10.pdf

Michael Nielsen. Chapter 6, "Deep Learning". In: *Neural Networks abd Deep Learning*. Determination Press, 2015.
http://neuralnetworksanddeeplearning.com/chap6.html

## 24.5 Summary

In this chapter, you have seen how to visualize the feature maps from a convolutional neural network and how it works to extract the feature maps.

Specifically, you learned:

- ▷ The structure of a typical convolutional neural networks
- ▷ What is the effect of the filter size to a convolutional layer
- ▷ What is the effect of stacking convolutional layers in a network

In the next chapter, you will see an example of using CNN to recognize images.

# Project: Handwritten Digit Recognition

25

A popular demonstration of the capability of deep learning techniques is object recognition in image data. The "hello world" of object recognition for machine learning and deep learning is the MNIST dataset for handwritten digit recognition. In this project, you will discover how to develop a deep learning model to achieve near state-of-the-art performance on the MNIST handwritten digit recognition task in Python using the Keras deep learning library. After completing this chapter, you will know:

- ▷ How to load the MNIST dataset in Keras
- ▷ How to develop and evaluate a baseline neural network model for the MNIST problem
- ▷ How to implement and evaluate a simple Convolutional Neural Network for MNIST
- ▷ How to implement a close to state-of-the-art deep learning model for MNIST

Let's get started.

**Note:** You may want to speed up the computation for this chapter by using GPU rather than CPU hardware, such as the process described in Appendix C. This is a suggestion, not a requirement. The code will work just fine on the CPU.

## 25.1 Description of the MNIST Handwritten Digit Recognition Problem

The MNIST problem is a dataset developed by Yann LeCun, Corinna Cortes, and Christopher Burges for evaluating machine learning models on the handwritten digit classification problem. The dataset was constructed from a number of scanned document datasets available from the National Institute of Standards and Technology (NIST). This is where the name for the dataset comes from, the Modified NIST or MNIST dataset.

Images of digits were taken from a variety of scanned documents, normalized in size, and centered. This makes it an excellent dataset for evaluating models, allowing the developer to focus on machine learning with minimal data cleaning or preparation required. Each image is a $28 \times 28$-pixel square (784 pixels total). A standard split of the dataset is used to evaluate

and compare models, where 60,000 images are used to train a model, and a separate set of 10,000 images are used to test it.

It is a digit recognition task. As such, there are ten digits (0 to 9) or ten classes to predict. Results are reported using prediction error, which is nothing more than the inverted classification accuracy. Excellent results achieve a prediction error of less than 1%. A state-of-the-art prediction error of approximately 0.2% can be achieved with large convolutional neural networks. There is a listing of the state-of-the-art results and links to the relevant papers on the MNIST and other datasets on Rodrigo Benenson's webpage.

## 25.2 Loading the MNIST Dataset in Keras

The Keras deep learning library provides a convenient method for loading the MNIST dataset. The dataset is downloaded automatically the first time this function is called and stored in your home directory in `~/.keras/datasets/mnist.npz` as an 11MB file. This is very handy for developing and testing deep learning models. To demonstrate how easy it is to load the MNIST dataset, first, write a little script to download and visualize the first four images in the training dataset.

```
# Plot ad hoc mnist instances
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
# load (downloaded if needed) the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# plot 4 images as gray scale
plt.subplot(221)
plt.imshow(X_train[0], cmap=plt.get_cmap('gray'))
plt.subplot(222)
plt.imshow(X_train[1], cmap=plt.get_cmap('gray'))
plt.subplot(223)
plt.imshow(X_train[2], cmap=plt.get_cmap('gray'))
plt.subplot(224)
plt.imshow(X_train[3], cmap=plt.get_cmap('gray'))
# show the plot
plt.show()
```

*Listing 25.1: Load the MNIST dataset in Keras*

You can see that downloading and loading the MNIST dataset is as easy as calling the `mnist.load_data()` function. Running the above example, you should see the image below.

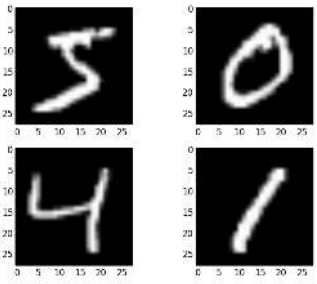

*Figure 25.1: Examples from the MNIST dataset*

## 25.3 Baseline Model with Multilayer Perceptrons

Do you really need a complex model like a convolutional neural network to get the best results with MNIST? You can get good results using a very simple neural network model with a single hidden layer. In this section, you will create a simple multilayer perceptron model that achieves an error rate of 1.74%. You will use this as a baseline for comparison to more complex convolutional neural network models. Let's start by importing the classes and functions you will need.

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.utils import to_categorical
...
```

*Listing 25.2: Import classes and functions*

Now, you can load the MNIST dataset using the Keras helper function.

```
...
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

*Listing 25.3: Load the MNIST dataset*

The training dataset is structured as a 3-dimensional array of instance, image width, and image height. For a multilayer perceptron model, you must reduce the images down into a vector of pixels. In this case, the $28 \times 28$-sized images will be 784 pixel input vectors. You can do this transform easily using the `reshape()` function on the NumPy array. The pixel values are integers, so they can be casted to floating point values and you can normalize them easily in the next step.

```
...
# flatten 28*28 images to a 784 vector for each image
num_pixels = X_train.shape[1] * X_train.shape[2]
X_train = X_train.reshape((X_train.shape[0], num_pixels)).astype('float32')
X_test = X_test.reshape((X_test.shape[0], num_pixels)).astype('float32')
```

*Listing 25.4: Prepare MNIST dataset for modeling*

The pixel values are grayscale between 0 and 255. It is almost always a good idea to perform some scaling of input values when using neural network models. Because the scale is well known and well behaved, you can very quickly normalize the pixel values to the range 0 and 1 by dividing each value by the maximum of 255.

```
...
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
```

*Listing 25.5: Normalize pixel values*

Finally, the output variable is an integer from 0 to 9. This is a multiclass classification problem. As such, it is good practice to use a one-hot encoding of the class values, transforming the vector of class integers into a binary matrix. You can easily do this using the built-in `tf.keras.utils.to_categorical()` helper function in Keras.

```
...
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
```

*Listing 25.6: One-hot encode the output variable*

You are now ready to create your simple neural network model. You will define your model in a function. This is handy if you want to extend the example later and try and get a better score.

```
...
# define baseline model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(num_pixels, input_shape=(num_pixels,),
                    kernel_initializer='normal', activation='relu'))
    model.add(Dense(num_classes, kernel_initializer='normal', activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
```

*Listing 25.7: Baseline model for recognition*

The model is a simple neural network with one hidden layer with the same number of neurons as there are inputs (784). A rectifier activation function is used for the neurons in the hidden layer. A softmax activation function is used on the output layer to turn the outputs into probability-like values and allow one class of the ten to be selected as the model's output prediction. Logarithmic loss is used as the loss function (called `categorical_crossentropy` in Keras), and the efficient ADAM gradient descent algorithm is used to learn the weights. A summary of the network structure is provided below:

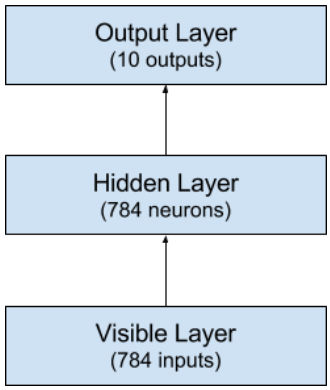


*Figure 25.2: Summary of multilayer perceptron network structure*

You can now fit and evaluate the model. The model is fit over ten epochs with updates every 200 images. The test data is used as the validation dataset, allowing you to see the skill of the model as it trains. A verbose value of 2 is used to reduce the output to one line for each training epoch. Finally, the test dataset is used to evaluate the model, and a classification error rate is printed.

```
...
# build the model
model = baseline_model()
# Fit the model
model.fit(X_train, y_train, epochs=10, batch_size=200,
          validation_data=(X_test, y_test), verbose=2)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Baseline Error: %.2f%%" % (100-scores[1]*100))
```

*Listing 25.8: Evaluate the baseline model*

After tying this all together, the complete code listing is provided below.

```
# Baseline MLP for MNIST dataset
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# flatten 28*28 images to a 784 vector for each image
num_pixels = X_train.shape[1] * X_train.shape[2]
X_train = X_train.reshape((X_train.shape[0], num_pixels)).astype('float32')
X_test = X_test.reshape((X_test.shape[0], num_pixels)).astype('float32')
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
# define baseline model
```

```
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(num_pixels, input_shape=(num_pixels,),
                    kernel_initializer='normal', activation='relu'))
    model.add(Dense(num_classes, kernel_initializer='normal', activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# build the model
model = baseline_model()
# Fit the model
model.fit(X_train, y_train, epochs=10, batch_size=200,
          validation_data=(X_test, y_test), verbose=2)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Baseline Error: %.2f%%" % (100-scores[1]*100))
```

*Listing 25.9: Multilayer perceptron model for MNIST problem*

Running the example might take a few minutes when you run it on a CPU. You should see the output below. This very simple network defined in very few lines of code achieves a respectable error rate of 2.3%.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Epoch 1/10
300/300 - 1s - loss: 0.2792 - accuracy: 0.9215 - val_loss: 0.1387 - val_accuracy: 0.9590 - 1s/epoch
Epoch 2/10
300/300 - 1s - loss: 0.1113 - accuracy: 0.9676 - val_loss: 0.0923 - val_accuracy: 0.9709 - 929ms/epo
Epoch 3/10
300/300 - 1s - loss: 0.0704 - accuracy: 0.9799 - val_loss: 0.0728 - val_accuracy: 0.9787 - 912ms/epo
Epoch 4/10
300/300 - 1s - loss: 0.0502 - accuracy: 0.9859 - val_loss: 0.0664 - val_accuracy: 0.9808 - 904ms/epo
Epoch 5/10
300/300 - 1s - loss: 0.0356 - accuracy: 0.9897 - val_loss: 0.0636 - val_accuracy: 0.9803 - 905ms/epo
Epoch 6/10
300/300 - 1s - loss: 0.0261 - accuracy: 0.9932 - val_loss: 0.0591 - val_accuracy: 0.9813 - 907ms/epo
Epoch 7/10
300/300 - 1s - loss: 0.0195 - accuracy: 0.9953 - val_loss: 0.0564 - val_accuracy: 0.9828 - 910ms/epo
Epoch 8/10
300/300 - 1s - loss: 0.0145 - accuracy: 0.9969 - val_loss: 0.0580 - val_accuracy: 0.9810 - 954ms/epo
Epoch 9/10
300/300 - 1s - loss: 0.0116 - accuracy: 0.9973 - val_loss: 0.0594 - val_accuracy: 0.9817 - 947ms/epo
Epoch 10/10
300/300 - 1s - loss: 0.0079 - accuracy: 0.9985 - val_loss: 0.0735 - val_accuracy: 0.9770 - 914ms/epo
Baseline Error: 2.30%
```

*Output 25.1: Sample output from evaluating the baseline model*

## 25.4 Simple Convolutional Neural Network for MNIST

Now that you have seen how to load the MNIST dataset and train a simple multilayer perceptron model on it, it is time to develop a more sophisticated convolutional neural network or CNN model. Keras does provide a lot of capability for creating convolutional neural networks. In this section, you will create a simple CNN for MNIST that demonstrates how to use all the aspects of a modern CNN implementation, including convolutional layers, pooling layers, and dropout layers.

The first step is to import the classes and functions needed.

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.utils import to_categorical
...
```

*Listing 25.10: Import classes and functions*

Next, you need to load the MNIST dataset and reshape it to be suitable for training a CNN. In Keras, the layers used for two-dimensional convolutions expect pixel values with the dimensions [*samples*][*width*][*height*][*channels*]. Note that you are forcing so-called channels-last ordering for consistency in this example. In the case of RGB, the last dimension channels would be 3 for the red, green, and blue components, and it would be like having three image inputs for every color image. In the case of MNIST, where the channels values are grayscale, the pixel dimension is set to 1.

```
...
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1).astype('float32')
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1).astype('float32')
```

*Listing 25.11: Load dataset and separate into train and test sets*

As before, it is a good idea to normalize the pixel values to the range 0 and 1 and one-hot encode the output variable.

```
...
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
```

*Listing 25.12: Normalize and one-hot encode data*

Next, define your neural network model. Convolutional neural networks are more complex than standard multilayer perceptrons, so you will start by using a simple structure that uses all the elements for state-of-the-art results. Below summarizes the network architecture.

1. The first hidden layer is a convolutional layer called a `Conv2D`. The layer has 32 feature maps, with the size of $5 \times 5$ and a rectifier activation function. This is the input layer that expects images with the structure outline above.
2. Next, we define a pooling layer that takes the max called `MaxPooling2D`. It is configured with a pool size of $2 \times 2$.
3. The next layer is a regularization layer using dropout called `Dropout`. It is configured to randomly exclude 20% of neurons in the layer in order to reduce overfitting.
4. Next is a layer that converts the 2D matrix data to a vector called `Flatten`. It allows the output to be processed by standard, fully connected layers.
5. Next is a fully connected layer with 128 neurons and a rectifier activation function is used.
6. Finally, the output layer has ten neurons for the ten classes and a softmax activation function to output probability-like predictions for each class.

As before, the model is trained using logarithmic loss and the ADAM gradient descent algorithm. A depiction of the network structure is provided below.

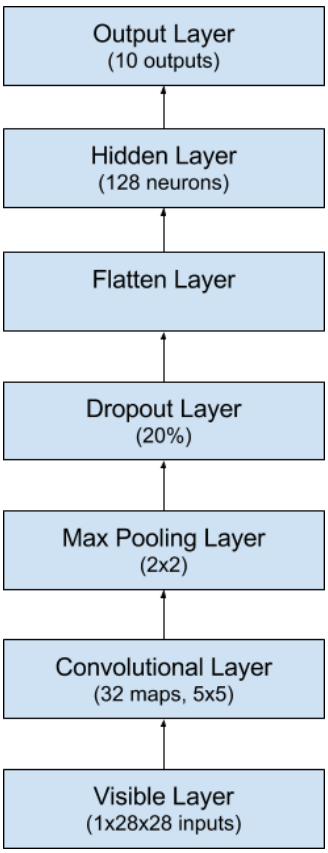


*Figure 25.3: Summary of convolutional neural network structure*

```
...
def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv2D(32, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
```

*Listing 25.13: Define and compile CNN model*

You evaluate the model the same way as before with the multilayer perceptron. The CNN is fit over ten epochs with a batch size of 200.

```
...
# build the model
model = baseline_model()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("CNN Error: %.2f%%" % (100-scores[1]*100))
```

*Listing 25.14: Fit and evaluate the CNN model*

After tying this all together, the complete example is listed below.

```
# Simple CNN for the MNIST Dataset
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.utils import to_categorical
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1)).astype('float32')
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1)).astype('float32')
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
# define a simple CNN model
```

```
def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv2D(32, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(MaxPooling2D())
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# build the model
model = baseline_model()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("CNN Error: %.2f%%" % (100-scores[1]*100))
```

*Listing 25.15: CNN model for MNIST problem*

After running the example, the accuracy of the training and validation test is printed for each epoch, and at the end, the classification error rate is printed.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Epochs may take about 45 seconds to run on the GPU (e.g., on AWS). You can see that the network achieves an error rate of 1.19%, which is better than our simple multilayer perceptron model above.

```
Epoch 1/10
300/300 [==============================] - 4s 12ms/step - loss: 0.2372 - accuracy: 0.9344 - val_loss
Epoch 2/10
300/300 [==============================] - 4s 13ms/step - loss: 0.0697 - accuracy: 0.9786 - val_loss
Epoch 3/10
300/300 [==============================] - 4s 13ms/step - loss: 0.0483 - accuracy: 0.9854 - val_loss
Epoch 4/10
300/300 [==============================] - 4s 13ms/step - loss: 0.0366 - accuracy: 0.9887 - val_loss
Epoch 5/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0300 - accuracy: 0.9909 - val_loss
Epoch 6/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0241 - accuracy: 0.9927 - val_loss
Epoch 7/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0210 - accuracy: 0.9932 - val_loss
Epoch 8/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0167 - accuracy: 0.9945 - val_loss
Epoch 9/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0142 - accuracy: 0.9956 - val_loss
Epoch 10/10
```

```
300/300 [==============================] - 4s 14ms/step - loss: 0.0114 - accuracy: 0.9966 - val_loss
CNN Error: 1.19%
```

*Output 25.2: Sample output from evaluating the CNN model*

## 25.5 Larger Convolutional Neural Network for MNIST

Now that you have seen how to create a simple CNN, let's take a look at a model capable of close to state-of-the-art results. You will import the classes and functions, then load and prepare the data the same as in the previous CNN example.

```
# Larger CNN for the MNIST Dataset
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.utils import to_categorical
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1)).astype('float32')
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1)).astype('float32')
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
...
```

*Listing 25.16: Import classes and functions for the larger CNN model*

This time you will define a large CNN architecture with additional convolutional, max pooling layers, and fully connected layers. The network topology can be summarized as follows:

1. Convolutional layer with 30 feature maps of size $5 \times 5$
2. Pooling layer taking the max over $2 \times 2$ patches
3. Convolutional layer with 15 feature maps of size $3 \times 3$
4. Pooling layer taking the max over $2 \times 2$ patches
5. Dropout layer with a probability of 20%
6. Flatten layer
7. Fully connected layer with 128 neurons and rectifier activation
8. Fully connected layer with 50 neurons and rectifier activation
9. Output layer

A depictions of this larger network structure is provided below.

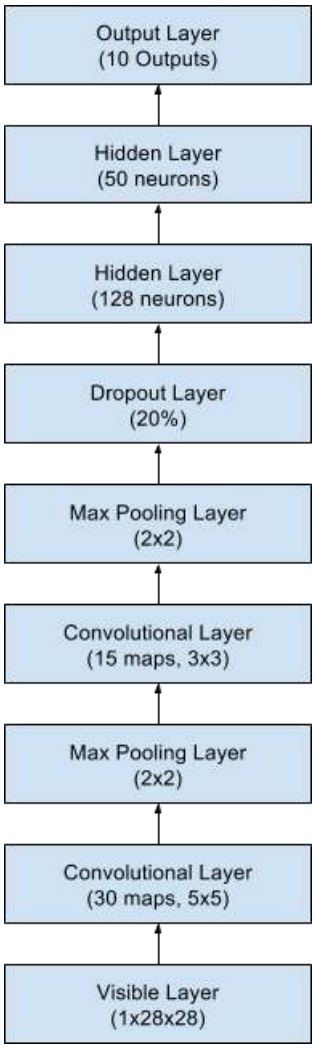


*Figure 25.4: Summary of the larger convolutional neural network structure*

```
...
# define the larger model
def larger_model():
    # create model
    model = Sequential()
    model.add(Conv2D(30, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Conv2D(15, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(50, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
```

*Listing 25.17: Defining the larger convolutional neural network*

Like the previous two experiments, the model is fit over ten epochs with a batch size of 200.

```
...
# build the model
model = larger_model()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Large CNN Error: %.2f%%" % (100-scores[1]*100))
```

*Listing 25.18: Fitting the larger CNN model*

After tying this all together, the complete example is listed below.

```
# Larger CNN for the MNIST Dataset
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.utils import to_categorical
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1)).astype('float32')
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1)).astype('float32')
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
# define the larger model
def larger_model():
    # create model
    model = Sequential()
    model.add(Conv2D(30, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(MaxPooling2D())
    model.add(Conv2D(15, (3, 3), activation='relu'))
    model.add(MaxPooling2D())
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(50, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# build the model
model = larger_model()
```

```
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Large CNN Error: %.2f%%" % (100-scores[1]*100))
```

*Listing 25.19: Larger CNN for the MNIST problem*

Running the example prints accuracy on the training and validation datasets of each epoch and a final classification error rate.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

The model takes about 100 seconds to run per epoch. This slightly larger model achieves a respectable classification error rate of 0.83%.

```
Epoch 1/10
300/300 [==============================] - 4s 14ms/step - loss: 0.4104 - accuracy: 0.8727 - val_loss
Epoch 2/10
300/300 [==============================] - 5s 15ms/step - loss: 0.1062 - accuracy: 0.9669 - val_loss
Epoch 3/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0771 - accuracy: 0.9765 - val_loss
Epoch 4/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0624 - accuracy: 0.9812 - val_loss
Epoch 5/10
300/300 [==============================] - 4s 15ms/step - loss: 0.0521 - accuracy: 0.9838 - val_loss
Epoch 6/10
300/300 [==============================] - 4s 15ms/step - loss: 0.0453 - accuracy: 0.9861 - val_loss
Epoch 7/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0415 - accuracy: 0.9866 - val_loss
Epoch 8/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0376 - accuracy: 0.9879 - val_loss
Epoch 9/10
300/300 [==============================] - 4s 14ms/step - loss: 0.0327 - accuracy: 0.9895 - val_loss
Epoch 10/10
300/300 [==============================] - 4s 15ms/step - loss: 0.0294 - accuracy: 0.9904 - val_loss
Large CNN Error: 0.90%
```

*Output 25.3: Sample output from evaluating the larger CNN model*

This is not an optimized network topology. Nor is it a reproduction of a network topology from a recent paper. There is a lot of opportunity for you to tune and improve upon this model. What is the best error rate score you can achieve?

## 25.6 Resources on MNIST

The MNIST dataset is very well studied. Below are some additional resources you might want to look into.

### Articles

Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. *The MNIST database of handwritten digits.*
http://yann.lecun.com/exdb/mnist/

Rodrigo Benenson. *What is the class of this image?* Classification datasets results, 2016.
https://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html

*Digit Recognizer: Learn computer vision fundamentals with the famous MNIST data.* Kaggle.
https://www.kaggle.com/c/digit-recognizer

Hubert Eichner. *Neural Net for Handwritten Digit Recognition in JavaScript.*
http://myselph.de/neuralNet.html

## 25.7 Summary

In this chapter, you discovered the MNIST handwritten digit recognition problem and deep learning models developed in Python using the Keras library that are capable of achieving excellent results. Working through this chapter, you learned:

- ▷ How to load the MNIST dataset in Keras and generate plots of the dataset
- ▷ How to reshape the MNIST dataset and develop a simple but well-performing multilayer perceptron model for the problem
- ▷ How to use Keras to create convolutional neural network models for MNIST
- ▷ How to develop and evaluate larger CNN models for MNIST capable of near world-class results

In the next chapter, you will learn some image preprocessing techniques to improve the model accuracy.

# Improve Model Accuracy with Image Augmentation

26

Data preparation is required when working with neural networks and deep learning models. Increasingly, data augmentation is also required on more complex object recognition tasks. This is because we can increase the variation of the input images so the network can be smarter from learning with more data. In this chapter, you will discover how to use data preparation and data augmentation with your image datasets when developing and evaluating deep learning models in Python with Keras. After reading this chapter, you will know:

- About the image augmentation API provided by Keras and how to use it with your models
- How to perform feature standardization
- How to perform ZCA whitening of your images
- How to augment data with random rotations, shifts and flips of images
- How to save augmented image data to disk

Let's get started.

## Overview

This chapter is divided into nine sections; they are:

- Keras Image Augmentation API
- Point of Comparison for Image Augmentation
- Feature Standardization
- ZCA Whitening
- Random Rotations
- Random Shifts
- Random Flips
- Saving Augmented Images to File
- Tips For Augmenting Image Data with Keras

## 26.1 Keras Image Augmentation API

Like the rest of Keras, the image augmentation API is simple and powerful. Keras provides the `ImageDataGenerator` class that defines the configuration for image data preparation and augmentation. This includes capabilities such as:

- ▷ Sample-wise standardization
- ▷ Feature-wise standardization
- ▷ ZCA whitening
- ▷ Random rotation, shifts, shear and flips
- ▷ Dimension reordering
- ▷ Save augmented images to disk

An augmented image generator can be created as follows:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator()
```

*Listing 26.1: Create a `ImageDataGenerator`*

Rather than performing the operations on your entire image dataset in memory, the API is designed to be iterated by the deep learning model fitting process, creating augmented image data for you just in time. This reduces your memory overhead but adds some additional time cost during model training. After you have created and configured your `ImageDataGenerator`, you must fit it on your data. This will calculate any statistics required to actually perform the transforms to your image data. You can do this by calling the `fit()` function on the data generator and passing it to your training dataset.

```
datagen.fit(train)
```

*Listing 26.2: Fit the `ImageDataGenerator`*

The data generator itself is, in fact, an iterator, returning batches of image samples when requested. You can configure the batch size and prepare the data generator and get batches of images by calling the `flow()` function.

```
X_batch, y_batch = datagen.flow(train, train, batch_size=32)
```

*Listing 26.3: Configure the batch size for the `ImageDataGenerator`*

Finally, you can make use of the data generator. Instead of calling the `fit()` function on your model, you must call the `fit_generator()` function and pass in the data generator and the desired length of an epoch as well as the total number of epochs on which to train.

```
fit_generator(datagen, samples_per_epoch=len(train), epochs=100)
```

*Listing 26.4: Fit a model using `ImageDataGenerator`*

You can learn more about the Keras image data generator API in the Keras documentation.

## 26.2 Point of Comparison for Image Augmentation

Now that you know how the image augmentation API in Keras works, let's look at some examples. We will use the MNIST handwritten digit recognition task in these examples (see Section 25.1). To begin, let's take a look at the first nine images in the training dataset.

```
# Plot images
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
# load dbata
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# create a grid of 3x3 images
fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
for i in range(3):
    for j in range(3):
        ax[i][j].imshow(X_train[i*3+j], cmap=plt.get_cmap("gray"))
# show the plot
plt.show()
```

*Listing 26.5: Load and plot the MNIST dataset*

Running this example provides the following image that you can use as a point of comparison with the image preparation and augmentation tasks in the examples below.

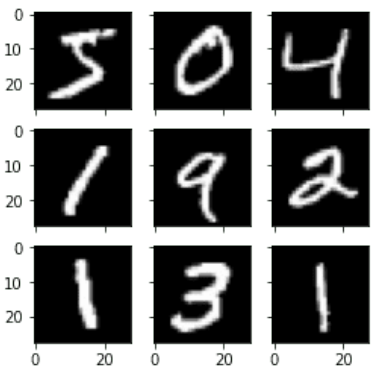

*Figure 26.1: Samples from the MNIST dataset.*

## 26.3 Feature Standardization

It is also possible to standardize pixel values across the entire dataset. This is called feature standardization and mirrors the type of standardization often performed for each column in a tabular dataset.

You can perform feature standardization by setting the `featurewise_center` and `featurewise_std_normalization` arguments to True on the `ImageDataGenerator` class. These are set to False by default. However, the recent version of Keras has a bug in the feature standardization so that the mean and standard deviation is calculated across all pixels. If you use the `fit()` function from the `ImageDataGenerator` class, you will see an image similar to the one above:

```
# Standardize images across the dataset, mean=0, stdev=1
from tensorflow.keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
# convert from int to float
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
# define data preparation
datagen = ImageDataGenerator(featurewise_center=True, featurewise_std_normalization=True)
# fit parameters from data
datagen.fit(X_train)
# configure batch size and retrieve one batch of images
for X_batch, y_batch in datagen.flow(X_train, y_train, batch_size=9, shuffle=False):
    print(X_batch.min(), X_batch.mean(), X_batch.max())
    # create a grid of 3x3 images
    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(X_batch[i*3+j].reshape(28,28), cmap=plt.get_cmap("gray"))
    # show the plot
    plt.show()
    break
```

*Listing 26.6: Standardize images across the dataset*

For example, the minimum, mean, and maximum values from the batch printed above are:

```
-0.42407447 -0.04093817 2.8215446
```

*Output 26.1: Minimum, mean, and maximum pixel value after standardization in Listing 26.6*

And the image displayed is as follows:

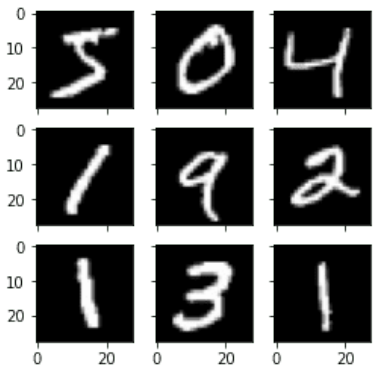


*Figure 26.2: Image from feature-wise standardization*

The workaround is to compute the feature standardization manually. Each pixel should have a separate mean and standard deviation, and it should be computed across different samples but independent from other pixels in the same sample. You just need to replace the `fit()` function with your own computation:

```
# Standardize images across the dataset, every pixel has mean=0, stdev=1
from tensorflow.keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
# convert from int to float
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
# define data preparation
datagen = ImageDataGenerator(featurewise_center=True, featurewise_std_normalization=True)
# fit parameters from data
datagen.mean = X_train.mean(axis=0)
datagen.std = X_train.std(axis=0)
# configure batch size and retrieve one batch of images
for X_batch, y_batch in datagen.flow(X_train, y_train, batch_size=9, shuffle=False):
    print(X_batch.min(), X_batch.mean(), X_batch.max())
    # create a grid of 3x3 images
    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(X_batch[i*3+j].reshape(28,28), cmap=plt.get_cmap("gray"))
    # show the plot
    plt.show()
    break
```

*Listing 26.7: Feature-wise standardizationn*

The minimum, mean, and maximum as printed now have a wider range:

```
-1.2742625 -0.028436039 17.46127
```

*Output 26.2: Minimum, mean. maximum pixel value after standardization in Listing 26.7*

Running this example, you can see that the effect is different, seemingly darkening and lightening different digits.

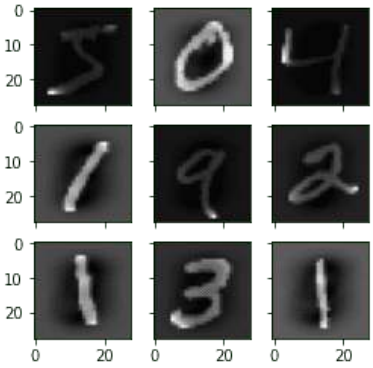

*Figure 26.3: MNIST images with standardized features*

## 26.4 ZCA Whitening

A whitening transform of an image is a linear algebraic operation that reduces the redundancy in the matrix of pixel images. Less redundancy in the image is intended to better highlight the structures and features in the image to the learning algorithm. Typically, image whitening is performed using the Principal Component Analysis (PCA) technique. More recently, an alternative called ZCA (learn more in Appendix A of Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*) shows better results in transformed images that keep all the original dimensions. And unlike PCA, the resulting transformed images still look like their originals. Precisely, whitening converts each image into a white noise vector, i.e., each element in the vector has zero mean unit standard derivation and is statistically independent of each other. You can perform a ZCA whitening transform by setting the `zca_whitening` argument to `True`. But due to the same issue as feature standardization, you must first zero-center your input data separately:

```
# ZCA Whitening
from tensorflow.keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
# convert from int to float
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
# define data preparation
datagen = ImageDataGenerator(featurewise_center=True,
                             featurewise_std_normalization=True,
                             zca_whitening=True)
# fit parameters from data
X_mean = X_train.mean(axis=0)
datagen.fit(X_train - X_mean)
# configure batch size and retrieve one batch of images
```

```
X_centered = X_train - X_mean
for X_batch, y_batch in datagen.flow(X_centered, y_train, batch_size=9, shuffle=False):
    print(X_batch.min(), X_batch.mean(), X_batch.max())
    # create a grid of 3x3 images
    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(X_batch[i*3+j].reshape(28,28), cmap=plt.get_cmap("gray"))
    # show the plot
    plt.show()
    break
```

*Listing 26.8: Example of ZCA whitening*

Running the example, you can see the same general structure in the images and how the outline of each digit has been highlighted.

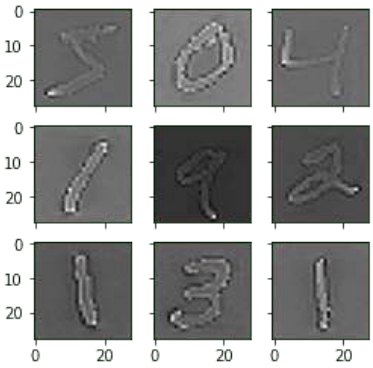

*Figure 26.4: ZCA whitening MNIST images*

## 26.5 Random Rotations

Sometimes images in your sample data may have varying and different rotations in the scene. You can train your model to better handle rotations of images by artificially and randomly rotating images from your dataset during training. The example below creates random rotations of the MNIST digits up to 90 degrees by setting the `rotation_range` argument.

```
# Random Rotations
from tensorflow.keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
# convert from int to float
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
# define data preparation
```

```
datagen = ImageDataGenerator(rotation_range=90)
# configure batch size and retrieve one batch of images
for X_batch, y_batch in datagen.flow(X_train, y_train, batch_size=9, shuffle=False):
    # create a grid of 3x3 images
    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(X_batch[i*3+j].reshape(28,28), cmap=plt.get_cmap("gray"))
    # show the plot
    plt.show()
    break
```

*Listing 26.9: Example of random image rotations*

Running the example, you can see that images have been rotated left and right up to a limit of 90 degrees. This is not helpful on this problem because the MNIST digits have a normalized orientation, but this transform might be of help when learning from photographs where the objects may have different orientations.

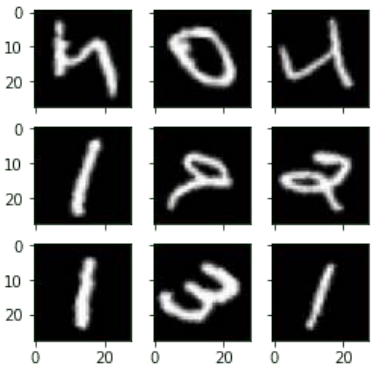

*Figure 26.5: Random rotations of MNIST images*

## 26.6 Random Shifts

Objects in your images may not be centered in the frame. They may be off-center in a variety of different ways. You can train your deep learning network to expect and currently handle off-center objects by artificially creating shifted versions of your training data. Keras supports separate horizontal and vertical random shifting of training data by the `width_shift_range` and `height_shift_range` arguments.

```
# Random Shifts
from tensorflow.keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
```

```
# convert from int to float
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
# define data preparation
shift = 0.2
datagen = ImageDataGenerator(width_shift_range=shift, height_shift_range=shift)
# configure batch size and retrieve one batch of images
for X_batch, y_batch in datagen.flow(X_train, y_train, batch_size=9, shuffle=False):
    # create a grid of 3x3 images
    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(X_batch[i*3+j].reshape(28,28), cmap=plt.get_cmap("gray"))
    # show the plot
    plt.show()
    break
```

*Listing 26.10: Example of random image shifts*

Running this example creates shifted versions of the digits. Again, this is not required for MNIST as the handwritten digits are already centered, but you can see how this might be useful on more complex problem domains.

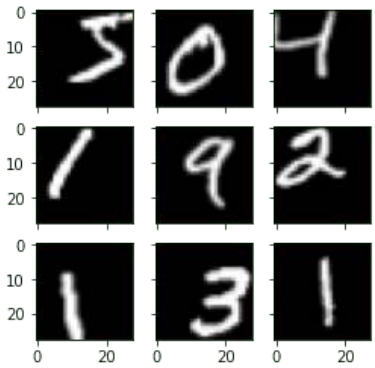

*Figure 26.6: Random shifted MNIST images*

## 26.7 Random Flips

Another augmentation to your image data that can improve performance on large and complex problems is to create random flips of images in your training data. Keras supports random flipping along both the vertical and horizontal axes using the `vertical_flip` and `horizontal_flip` arguments.

```
# Random Flips
from tensorflow.keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
# convert from int to float
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
# define data preparation
datagen = ImageDataGenerator(horizontal_flip=True, vertical_flip=True)
# configure batch size and retrieve one batch of images
for X_batch, y_batch in datagen.flow(X_train, y_train, batch_size=9, shuffle=False):
    # create a grid of 3x3 images
    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(X_batch[i*3+j].reshape(28,28), cmap=plt.get_cmap("gray"))
    # show the plot
    plt.show()
    break
```

*Listing 26.11: Example of random image flips*

Running this example, you can see flipped digits. Flipping digits in MNIST is not useful as they will always have the correct left and right orientation, but this may be useful for problems with photographs of objects in a scene that can have a varied orientation.

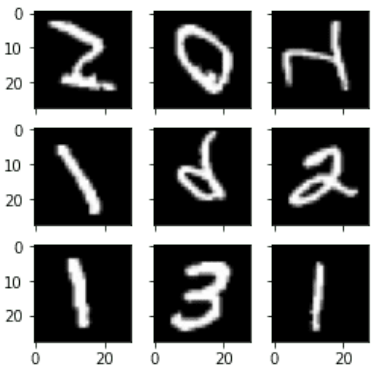


*Figure 26.7: Randomly flipped MNIST images*

## 26.8 Saving Augmented Images to File

The data preparation and augmentation are performed just-in-time by Keras. This is efficient in terms of memory, but you may require the exact images used during training. For example, perhaps you would like to use them with a different software package later or only generate them once and use them on multiple different deep learning models or configurations.

Keras allows you to save the images generated during training. The directory, filename prefixes, and image file type can be specified to the `flow()` function before training. Then, during training, the generated images will be written to the file. The example below demonstrates this and writes nine images to a `images` subdirectory (you need to create this directory first) with the prefix `aug` and the file type of PNG.

```
# Save augmented images to file
from tensorflow.keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
# convert from int to float
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
# define data preparation
datagen = ImageDataGenerator(horizontal_flip=True, vertical_flip=True)
# configure batch size and retrieve one batch of images
for X_batch, y_batch in datagen.flow(X_train, y_train, batch_size=9, shuffle=False,
                                     save_to_dir='images', save_prefix='aug',
                                     save_format='png'):
    # create a grid of 3x3 images
    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(4,4))
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(X_batch[i*3+j].reshape(28,28), cmap=plt.get_cmap("gray"))
    # show the plot
    plt.show()
    break
```

*Listing 26.12: Save augmented images to file*

Running the example, you can see that images are only written when they are generated.

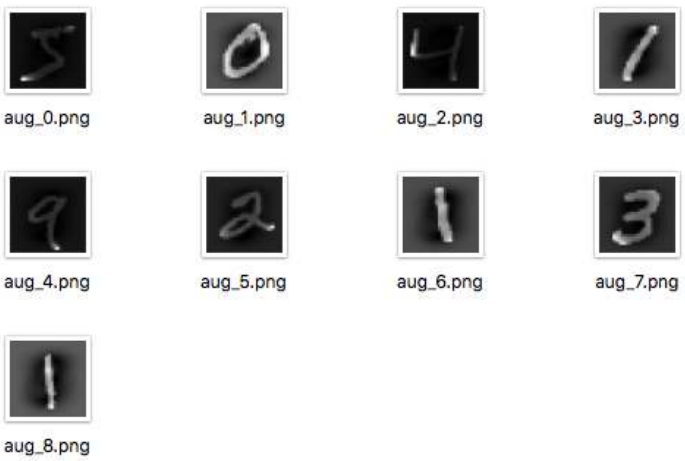


*Figure 26.8: Augmented MNIST images saved to file*

## 26.9 Tips for Augmenting Image Data with Keras

Image data is unique in that you can review the data and transformed copies of the data and quickly get an idea of how the inputs may perceived it.

Below are some tips for getting the most from image data preparation and augmentation for deep learning.

- ▷ **Review Dataset**. Take some time to review your dataset in great detail. Look at the images. Take note of image preparation and augmentations that might benefit the training process of your model, such as the need to handle different shifts, rotations, or flips of objects in the scene.
- ▷ **Review Augmentations**. Review sample images after the augmentation has been performed. It is one thing to intellectually know what image transforms you are using; it is a very different thing to look at examples. Review images both with individual augmentations you are using as well as the full set of augmentations you plan to use in aggregate. You may see ways to simplify or further enhance your model training process.
- ▷ **Evaluate a Suite of Transforms**. Try more than one image data preparation and augmentation scheme. Often you can be surprised by the results of a data preparation scheme you did not think would be beneficial.

## 26.10 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

### APIs

*tf.keras.preprocessing.image.ImageDataGenerator*. Tensorflow API.
https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator

*Image data preprocessing*. Keras API reference.
https://keras.io/api/preprocessing/image/

### Articles

Alex Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Tech Report TR-2009. University of Toronto, Apr. 2009.
https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

*Whitening transformation*. Wikipedia.
https://en.wikipedia.org/wiki/Whitening_transformation

*PCA Whitening*. Unsupervised Feature Learning and Deep Learning Tutorial.
http://ufldl.stanford.edu/tutorial/unsupervised/PCAWhitening/

Alex Krizhevsky. *The CIFAR-10 dataset*.
https://www.cs.toronto.edu/~kriz/cifar.html

## 26.11 Summary

In this chapter, you discovered image data preparation and augmentation.

You discovered a range of techniques you can use easily in Python with Keras for deep learning models. You learned about:

- The `ImageDataGenerator` API in Keras for generating transformed images just-in-time
- Sample-wise and feature-wise pixel standardization
- The ZCA whitening transform
- Random rotations, shifts and flips of images
- How to save transformed images to file for later reuse

In the next chapter, you will see more ways to do image augmentation.

# Image Augmentation with Keras Preprocessing Layers and tf.image

27

When you work on a machine learning problem related to images, not only do you need to collect some images as training data, but you also need to employ augmentation to create variations in the image. It is especially true for more complex object recognition problems.

There are many ways for image augmentation. You may use some external libraries or write your own functions for that. There are some modules in TensorFlow and Keras for augmentation too. In this chapter, you will discover how you can use the Keras preprocessing layer as well as the `tf.image` module in TensorFlow for image augmentation. After reading this chapter, you will know:

- ▷ What are the Keras preprocessing layers, and how to use them
- ▷ What are the functions provided by the `tf.image` module for image augmentation
- ▷ How to use augmentation together with the `tf.data` dataset

Let's get started.

## Overview

This chapter is divided into five sections; they are:

- ▷ Getting Images
- ▷ Visualizing the Images
- ▷ Keras Preprocessing Layers
- ▷ Using tf.image API for Augmentation
- ▷ Using Preprocessing Layers in Neural Networks

## 27.1 Getting Images

Before you see how you can do augmentation, you need to get the images. Ultimately, you need the images to be represented as arrays, for example, in $H \times W \times 3$ in 8-bit integers for the RGB pixel value. There are many ways to get the images. Some can be downloaded as a ZIP file.

If you're using TensorFlow, you may get some image datasets from the `tensorflow_datasets` library.

In this chapter, you will use the citrus leaves images, which is a small dataset of less than 100MB. It can be downloaded from `tensorflow_datasets` as follows:

```
import tensorflow_datasets as tfds
ds, meta = tfds.load('citrus_leaves', with_info=True, split='train', shuffle_files=True)
```

*Listing 27.1: Loading the citrus leaves dataset*

Running this code the first time will download the image dataset into your computer with the following output:

```
Downloading and preparing dataset 63.87 MiB (download: 63.87 MiB, generated: 37.89 MiB, total:
↪  101.76 MiB) to ~/tensorflow_datasets/citrus_leaves/0.1.2...
Extraction completed...: 100%|████████████████████████████| 1/1 [00:06<00:00,  6.54s/ file]
Dl Size...: 100%|████████████████████████████████████████| 63/63 [00:06<00:00,  9.63 MiB/s]
Dl Completed...: 100%|███████████████████████████████████| 1/1 [00:06<00:00,  6.54s/ url]
Dataset citrus_leaves downloaded and prepared to ~/tensorflow_datasets/citrus_leaves/0.1.2.
↪  Subsequent calls will reuse this data.
```

*Output 27.1: Downloading the citrus leaves dataset the first time*

The function above returns the images as a `tf.data` dataset object and the metadata. This is a classification dataset. You can print the training labels with the following:

```
...
for i in range(meta.features['label'].num_classes):
    print(meta.features['label'].int2str(i))
```

*Listing 27.2: Print the classification labels of the citrus leaves dataset*

This prints:

```
Black spot
canker
greening
healthy
```

*Output 27.2: Classification labels of the citrus leaves dataset*

If you run this code again at a later time, you will reuse the downloaded image. But the other way to load the downloaded images into a `tf.data` dataset is to use the `image_dataset_from_directory()` function.

As you can see from the screen output above, the dataset is downloaded into the directory `~/tensorflow_datasets`. If you look at the directory, you see the directory structure as follows:

```
.../Citrus/Leaves
├── Black spot
├── Melanose
├── canker
```

```
├── greening
└── healthy
```

*Output 27.3: Directory structure of the downloaded dataset*

The directories are the labels, and the images are files stored under their corresponding directory. You can let the function to read the directory recursively into a dataset:

```
import tensorflow as tf
from tensorflow.keras.utils import image_dataset_from_directory

# set to fixed image size 256x256
PATH = ".../Citrus/Leaves"
ds = image_dataset_from_directory(PATH,
                                  validation_split=0.2, subset="training",
                                  image_size=(256,256), interpolation="bilinear",
                                  crop_to_aspect_ratio=True,
                                  seed=42, shuffle=True, batch_size=32)
```

*Listing 27.3: Load the citrus leaves dataset from local hard disk*

You may want to set `batch_size=None` if you do not want the dataset to be batched. Usually, you want the dataset to be batched for training a neural network model.

## 27.2 Visualizing the Images

It is important to visualize the augmentation result, so you can verify the augmentation result is what we want it to be. You can use Matplotlib for this. In Matplotlib, you have the `imshow()` function to display an image. However, for the image to be displayed correctly, the image should be presented as an array of 8-bit unsigned integers (uint8). Given that you have a dataset created using `image_dataset_from_directory()`. You can get the first batch (of 32 images) and display a few of them using `imshow()`, as follows:

```
...
import matplotlib.pyplot as plt

fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(5,5))

for images, labels in ds.take(1):
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(images[i*3+j].numpy().astype("uint8"))
            ax[i][j].set_title(ds.class_names[labels[i*3+j]])
plt.show()
```

*Listing 27.4: Show example images from a dataset*

Here, you see a display of nine images in a grid, labeled with their corresponding classification label, using `ds.class_names`. The images should be converted to NumPy array in uint8 for display. This code displays an image like the following:

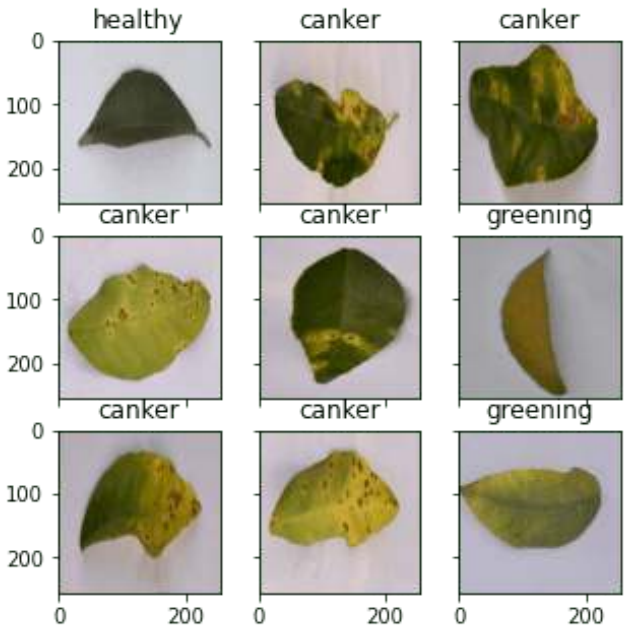


*Figure 27.1: Example images from the citrus leaves dataset*

The complete code from loading the image to display is as follows:

```
from tensorflow.keras.utils import image_dataset_from_directory
import matplotlib.pyplot as plt

# use image_dataset_from_directory() to load images, with image size scaled to 256x256
PATH='.../Citrus/Leaves'  # modify to your path
ds = image_dataset_from_directory(PATH,
                                  validation_split=0.2, subset="training",
                                  image_size=(256,256), interpolation="mitchellcubic",
                                  crop_to_aspect_ratio=True,
                                  seed=42, shuffle=True, batch_size=32)

# Take one batch from dataset and display the images
fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(5,5))

for images, labels in ds.take(1):
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(images[i*3+j].numpy().astype("uint8"))
            ax[i][j].set_title(ds.class_names[labels[i*3+j]])
plt.show()
```

*Listing 27.5: Loading images from disk and display a few samples*

Note that if you're using `tensorflow_datasets` to get the image, the samples are presented as a dictionary instead of a tuple of (image,label). You should change your code slightly to the following:

```
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt

# use tfds.load() or image_dataset_from_directory() to load images
ds, meta = tfds.load('citrus_leaves', with_info=True, split='train', shuffle_files=True)
ds = ds.batch(32)

# Take one batch from dataset and display the images
fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(5,5))

for sample in ds.take(1):
    images, labels = sample["image"], sample["label"]
    for i in range(3):
        for j in range(3):
            ax[i][j].imshow(images[i*3+j].numpy().astype("uint8"))
            ax[i][j].set_title(meta.features['label'].int2str(labels[i*3+j]))
plt.show()
```

*Listing 27.6: Loading images from `tensorflow_datasets` and display a few samples*

**Note:** For the rest of this chapter, assume the dataset is created using `image_dataset_from_directory()`. You may need to tweak the code slightly if your dataset is created differently.

## 27.3 Keras Preprocessing Layers

Keras comes with many neural network layers, such as convolution layers, that you need to train. There are also layers with no parameters to train, such as flatten layers to convert an array like an image into a vector.

The preprocessing layers in Keras are specifically designed to use in the early stages of a neural network. You can use them for image preprocessing, such as to resize or rotate the image or adjust the brightness and contrast. While the preprocessing layers are supposed to be part of a larger neural network, you can also use them as functions. Below is how you can use the resizing layer as a function to transform some images and display them side-by-side with the original:

```
...

# create a resizing layer
out_height, out_width = 128,256
resize = tf.keras.layers.Resizing(out_height, out_width)

# show original vs resized
fig, ax = plt.subplots(2, 3, figsize=(6,4))

for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
```

```
        ax[0][i].set_title("original")
        # resize
        ax[1][i].imshow(resize(images[i]).numpy().astype("uint8"))
        ax[1][i].set_title("resize")
plt.show()
```

*Listing 27.7: Resize images*

The images are in $256 \times 256$ pixels, and the resizing layer will make them into $256 \times 128$ pixels. The output of the above code is as follows:

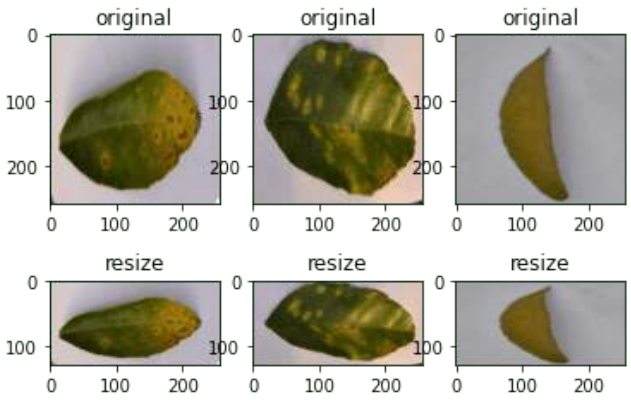


*Figure 27.2: Example of images and their resize transform*

Since the resizing layer is a function, you can chain them to the dataset itself. For example,

```
...
def augment(image, label):
    return resize(image), label

resized_ds = ds.map(augment)

for image, label in resized_ds:
   ...
```

*Listing 27.8: Attach resize layer into dataset*

The dataset `ds` has samples in the form of `(image, label)`. Hence you created a function that takes in such tuple and preprocesses the image with the resizing layer. You then assigned this function as an argument for the `map()` in the dataset. When you draw a sample from the new dataset created with the `map()` function, the image will be a transformed one.

There are more preprocessing layers available. Some are demonstrated below.

The examples below are tested with TensorFlow 2.9. Some of the preprocessing functions may not be availble in earlier version of TensorFlow.

As we saw above, we can resize the image. We can also randomly enlarge or shrink the height or width of an image. Similarly, we can zoom in or zoom out on an image. Below is

an example to manipulate the image size in various ways for a maximum of 30% increase or decrease:

```
...

# Create preprocessing layers
out_height, out_width = 128,256
resize = tf.keras.layers.Resizing(out_height, out_width)
height = tf.keras.layers.RandomHeight(0.3)
width = tf.keras.layers.RandomWidth(0.3)
zoom = tf.keras.layers.RandomZoom(0.3)

# Visualize images and augmentations
fig, ax = plt.subplots(5, 3, figsize=(6,14))

for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # resize
        ax[1][i].imshow(resize(images[i]).numpy().astype("uint8"))
        ax[1][i].set_title("resize")
        # height
        ax[2][i].imshow(height(images[i]).numpy().astype("uint8"))
        ax[2][i].set_title("height")
        # width
        ax[3][i].imshow(width(images[i]).numpy().astype("uint8"))
        ax[3][i].set_title("width")
        # zoom
        ax[4][i].imshow(zoom(images[i]).numpy().astype("uint8"))
        ax[4][i].set_title("zoom")
plt.show()
```

*Listing 27.9: Transform images by resize, adjusting height, adjusting width, and zoom*

This code shows images as in Figure 27.3. While you specified a fixed dimension in resize, you have a random amount of manipulation in other augmentations.

You can also do flipping, rotation, cropping, and geometric translation using preprocessing layers:

```
...
# Create preprocessing layers
flip = tf.keras.layers.RandomFlip("horizontal_and_vertical") # or "horizontal", "vertical"
rotate = tf.keras.layers.RandomRotation(0.2)
crop = tf.keras.layers.RandomCrop(out_height, out_width)
translation = tf.keras.layers.RandomTranslation(height_factor=0.2, width_factor=0.2)

# Visualize augmentations
fig, ax = plt.subplots(5, 3, figsize=(6,14))

for images, labels in ds.take(1):
    for i in range(3):
```

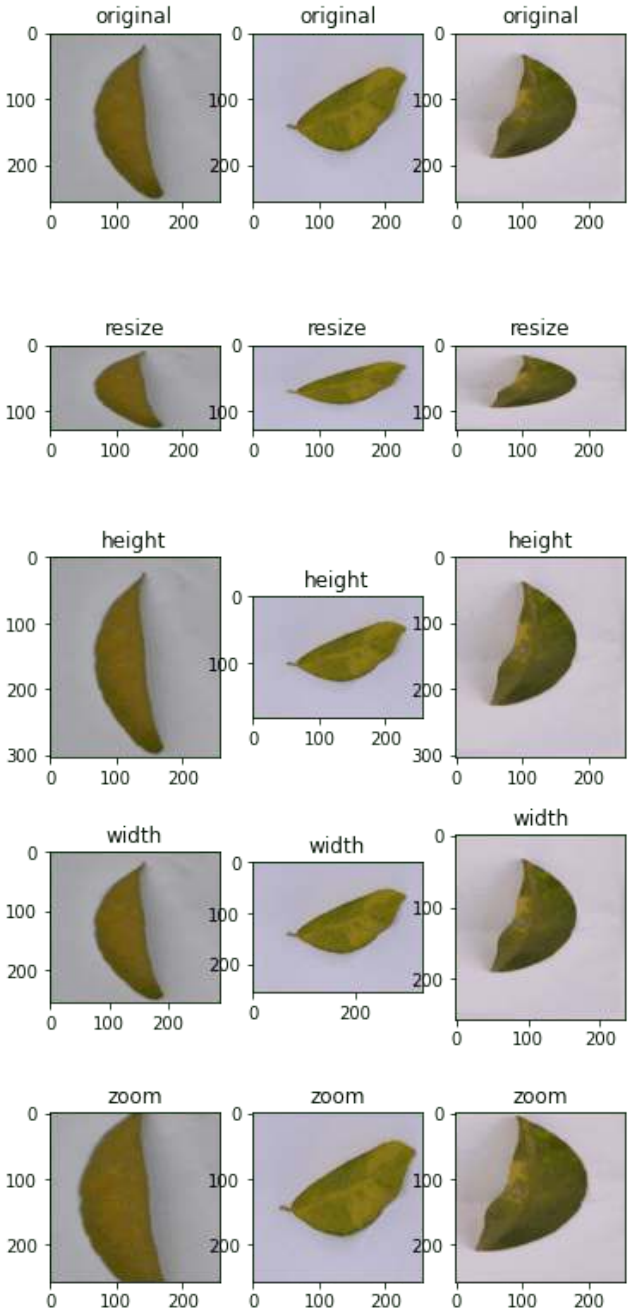


*Figure 27.3: Example transformation from Listing 27.9*

```
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # flip
        ax[1][i].imshow(flip(images[i]).numpy().astype("uint8"))
        ax[1][i].set_title("flip")
        # crop
        ax[2][i].imshow(crop(images[i]).numpy().astype("uint8"))
        ax[2][i].set_title("crop")
        # translation
        ax[3][i].imshow(translation(images[i]).numpy().astype("uint8"))
```

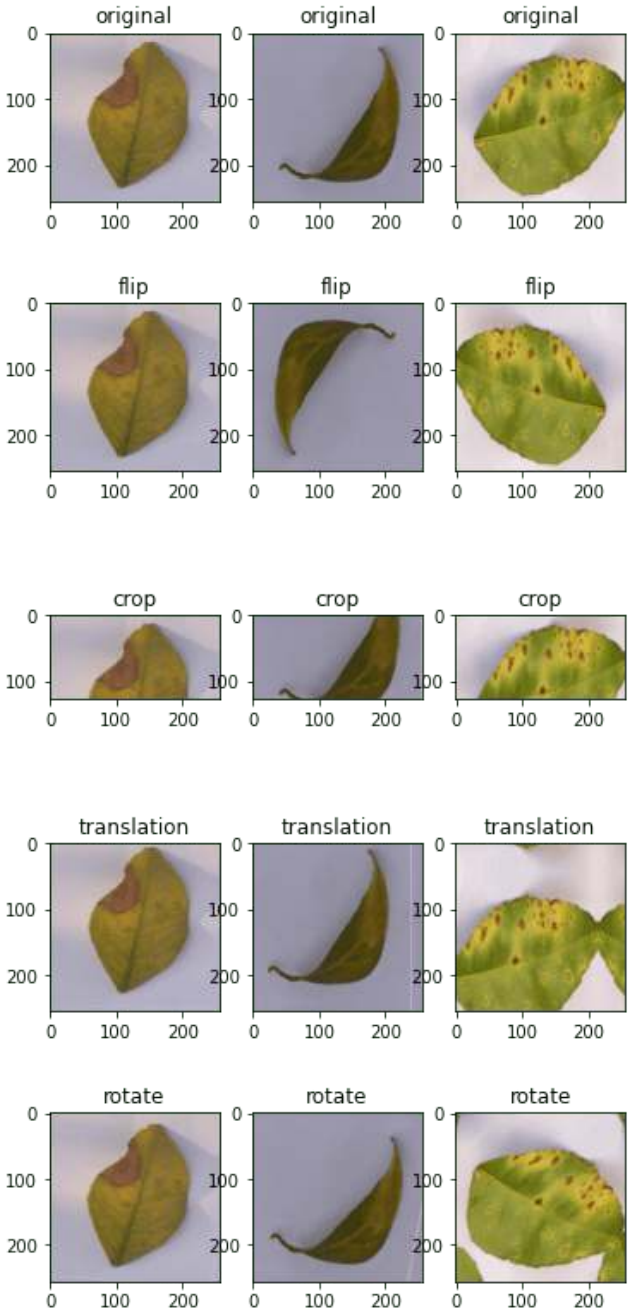


*Figure 27.4: Example transformation from Listing 27.10*

```
        ax[3][i].set_title("translation")
        # rotate
        ax[4][i].imshow(rotate(images[i]).numpy().astype("uint8"))
        ax[4][i].set_title("rotate")
plt.show()
```

*Listing 27.10: Transform images by flipping, cropping, translation, and rotation*

This code shows the images as in Figure 27.4.

And finally, you can do augmentations on color adjustments as well:

```
...
brightness = tf.keras.layers.RandomBrightness([-0.8,0.8])
contrast = tf.keras.layers.RandomContrast(0.2)

# Visualize augmentation
fig, ax = plt.subplots(3, 3, figsize=(6,7))

for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # brightness
        ax[1][i].imshow(brightness(images[i]).numpy().astype("uint8"))
        ax[1][i].set_title("brightness")
        # contrast
        ax[2][i].imshow(contrast(images[i]).numpy().astype("uint8"))
        ax[2][i].set_title("contrast")
plt.show()
```

*Listing 27.11: Transform images by adjusting brightness and contrast*

This shows the images as follows:

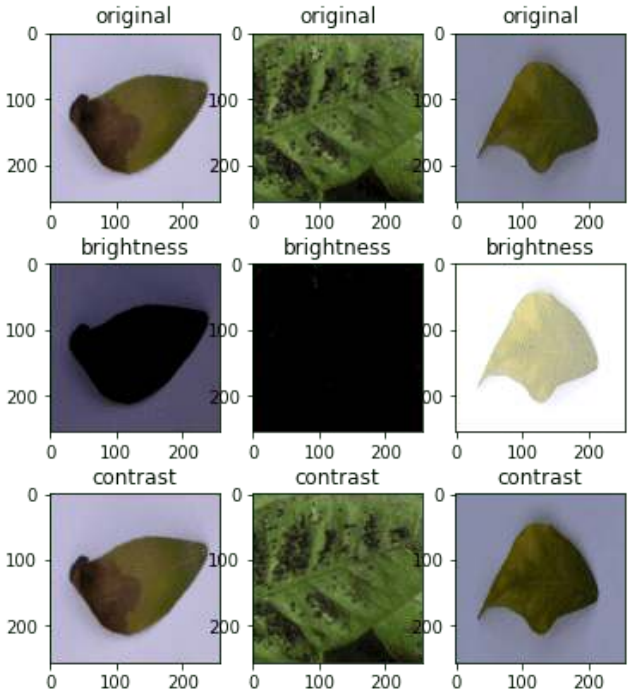


*Figure 27.5: Example transformation from Listing 27.11*

For completeness, below is the code to display the result of various augmentations:

```
from tensorflow.keras.utils import image_dataset_from_directory
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
# use image_dataset_from_directory() to load images, with image size scaled to 256x256
PATH='.../Citrus/Leaves'  # modify to your path
ds = image_dataset_from_directory(PATH,
                                  validation_split=0.2, subset="training",
                                  image_size=(256,256), interpolation="mitchellcubic",
                                  crop_to_aspect_ratio=True,
                                  seed=42, shuffle=True, batch_size=32)

# Create preprocessing layers
out_height, out_width = 128,256
resize = tf.keras.layers.Resizing(out_height, out_width)
height = tf.keras.layers.RandomHeight(0.3)
width = tf.keras.layers.RandomWidth(0.3)
zoom = tf.keras.layers.RandomZoom(0.3)

flip = tf.keras.layers.RandomFlip("horizontal_and_vertical")
rotate = tf.keras.layers.RandomRotation(0.2)
crop = tf.keras.layers.RandomCrop(out_height, out_width)
translation = tf.keras.layers.RandomTranslation(height_factor=0.2, width_factor=0.2)

brightness = tf.keras.layers.RandomBrightness([-0.8,0.8])
contrast = tf.keras.layers.RandomContrast(0.2)

# Visualize images and augmentations
fig, ax = plt.subplots(5, 3, figsize=(6,14))
for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # resize
        ax[1][i].imshow(resize(images[i]).numpy().astype("uint8"))
        ax[1][i].set_title("resize")
        # height
        ax[2][i].imshow(height(images[i]).numpy().astype("uint8"))
        ax[2][i].set_title("height")
        # width
        ax[3][i].imshow(width(images[i]).numpy().astype("uint8"))
        ax[3][i].set_title("width")
        # zoom
        ax[4][i].imshow(zoom(images[i]).numpy().astype("uint8"))
        ax[4][i].set_title("zoom")
plt.show()

fig, ax = plt.subplots(5, 3, figsize=(6,14))
for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # flip
        ax[1][i].imshow(flip(images[i]).numpy().astype("uint8"))
        ax[1][i].set_title("flip")
        # crop
        ax[2][i].imshow(crop(images[i]).numpy().astype("uint8"))
        ax[2][i].set_title("crop")
```

```
        # translation
        ax[3][i].imshow(translation(images[i]).numpy().astype("uint8"))
        ax[3][i].set_title("translation")
        # rotate
        ax[4][i].imshow(rotate(images[i]).numpy().astype("uint8"))
        ax[4][i].set_title("rotate")
plt.show()

fig, ax = plt.subplots(3, 3, figsize=(6,7))
for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # brightness
        ax[1][i].imshow(brightness(images[i]).numpy().astype("uint8"))
        ax[1][i].set_title("brightness")
        # contrast
        ax[2][i].imshow(contrast(images[i]).numpy().astype("uint8"))
        ax[2][i].set_title("contrast")
plt.show()
```

*Listing 27.12: Various augmentations using Keras preprocessing layers*

Finally, it is important to point out that most neural network models can work better if the input images are scaled. While we usually use an 8-bit unsigned integer for the pixel values in an image (e.g., for display using `imshow()` as above), a neural network prefers the pixel values to be between 0 and 1 or between $-1$ and $+1$. This can be done with preprocessing layers too. Below is how you can update one of the examples above to add the scaling layer into the augmentation:

```
...
out_height, out_width = 128,256
resize = tf.keras.layers.Resizing(out_height, out_width)
rescale = tf.keras.layers.Rescaling(1/127.5, offset=-1)  # rescale pixel values to [-1,1]

def augment(image, label):
    return rescale(resize(image)), label

rescaled_resized_ds = ds.map(augment)

for image, label in rescaled_resized_ds:
   ...
```

*Listing 27.13: Rescaling pixel values of an image*

## 27.4 Using tf.image API for Augmentation

Besides the preprocessing layer, the `tf.image` module also provides some functions for augmentation. Unlike the preprocessing layer, these functions are intended to be used in a user-defined function and assigned to a dataset using `map()` as we saw above.

The functions provided by `tf.image` are not duplicates of the preprocessing layers, although there is some overlap. Below is an example of using the `tf.image` functions to resize and crop images:

```
...

fig, ax = plt.subplots(5, 3, figsize=(6,14))

for images, labels in ds.take(1):
    for i in range(3):
        # original
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # resize
        h = int(256 * tf.random.uniform([], minval=0.8, maxval=1.2))
        w = int(256 * tf.random.uniform([], minval=0.8, maxval=1.2))
        ax[1][i].imshow(tf.image.resize(images[i], [h,w]).numpy().astype("uint8"))
        ax[1][i].set_title("resize")
        # crop
        y, x, h, w = (128 * tf.random.uniform((4,))).numpy().astype("uint8")
        ax[2][i].imshow(tf.image.crop_to_bounding_box(images[i], y, x, h, w)
                        .numpy().astype("uint8"))
        ax[2][i].set_title("crop")
        # central crop
        x = tf.random.uniform([], minval=0.4, maxval=1.0)
        ax[3][i].imshow(tf.image.central_crop(images[i], x).numpy().astype("uint8"))
        ax[3][i].set_title("central crop")
        # crop to (h,w) at random offset
        h, w = (256 * tf.random.uniform((2,))).numpy().astype("uint8")
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
        ax[4][i].imshow(tf.image.stateless_random_crop(images[i], [h,w,3], seed)
                        .numpy().astype("uint8"))
        ax[4][i].set_title("random crop")
plt.show()
```

*Listing 27.14: Transform images by resizing and cropping using `tf.image` API*

The output of the above code is in Figure 27.6.

While the display of images matches what you might expect from the code, the use of `tf.image` functions is quite different from that of the preprocessing layers. Every `tf.image` function is different. Therefore, you can see the `crop_to_bounding_box()` function takes pixel coordinates, but the `central_crop()` function assumes a fraction ratio as the argument.

These functions are also different in the way randomness is handled. Some of these functions do not assume random behavior. Therefore, the random resize should have the exact output size generated using a random number generator separately before calling the resize function. Some other functions, such as `stateless_random_crop()`, can do augmentation randomly, but a pair of random seeds in the `int32` needs to be specified explicitly.

To continue the example, there are the functions for flipping an image and extracting the Sobel edges:

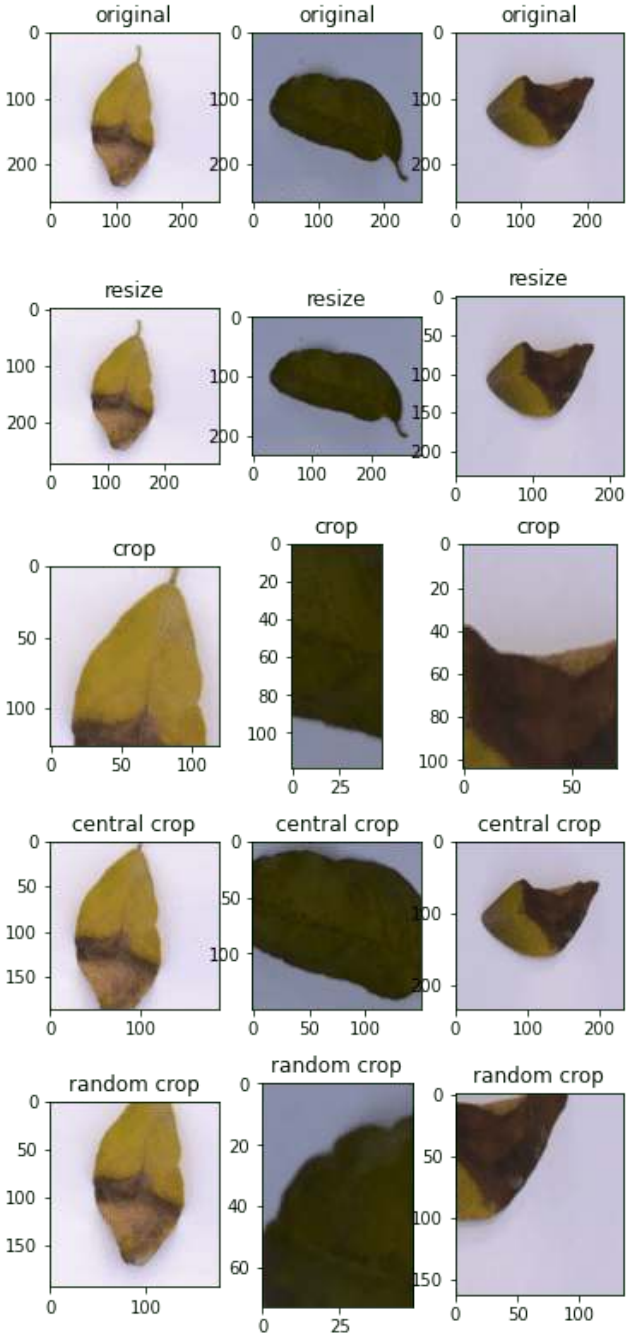


*Figure 27.6: Example transformation from Listing 27.14*

.

```
...
fig, ax = plt.subplots(5, 3, figsize=(6,14))

for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # flip
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
        ax[1][i].imshow(tf.image.stateless_random_flip_left_right(images[i], seed)
                        .numpy().astype("uint8"))
        ax[1][i].set_title("flip left-right")
        # flip
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
        ax[2][i].imshow(tf.image.stateless_random_flip_up_down(images[i], seed)
                        .numpy().astype("uint8"))
        ax[2][i].set_title("flip up-down")
        # sobel edge
        sobel = tf.image.sobel_edges(images[i:i+1])
        ax[3][i].imshow(sobel[0, ..., 0].numpy().astype("uint8"))
        ax[3][i].set_title("sobel y")
        # sobel edge
        ax[4][i].imshow(sobel[0, ..., 1].numpy().astype("uint8"))
        ax[4][i].set_title("sobel x")
plt.show()
```

*Listing 27.15: Transform images by flipping and sobel transform*

This shows the image in Figure 27.7.

And the following are the functions to manipulate the brightness, contrast, and colors:

```
...
fig, ax = plt.subplots(5, 3, figsize=(6,14))

for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # brightness
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
        ax[1][i].imshow(tf.image.stateless_random_brightness(images[i], 0.3, seed)
                        .numpy().astype("uint8"))
        ax[1][i].set_title("brightness")
        # contrast
        ax[2][i].imshow(tf.image.stateless_random_contrast(images[i], 0.7, 1.3, seed)
                        .numpy().astype("uint8"))
        ax[2][i].set_title("contrast")
        # saturation
        ax[3][i].imshow(tf.image.stateless_random_saturation(images[i], 0.7, 1.3, seed)
                        .numpy().astype("uint8"))
        ax[3][i].set_title("saturation")
        # hue
        ax[4][i].imshow(tf.image.stateless_random_hue(images[i], 0.3, seed)
```

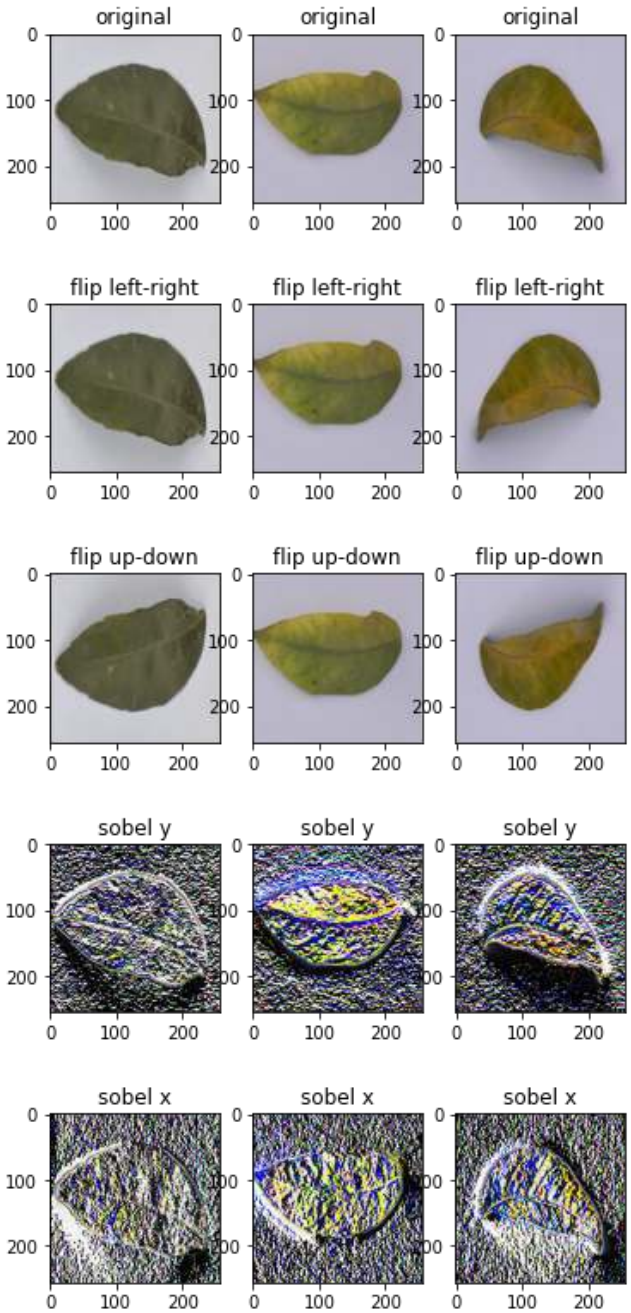


*Figure 27.7: Example transformation from Listing 27.15*

```
                        .numpy().astype("uint8"))
        ax[4][i].set_title("hue")
plt.show()
```

*Listing 27.16: Transform images by adjusting brightness, contrast, saturation, and hue*

This code shows the image in Figure 27.8.

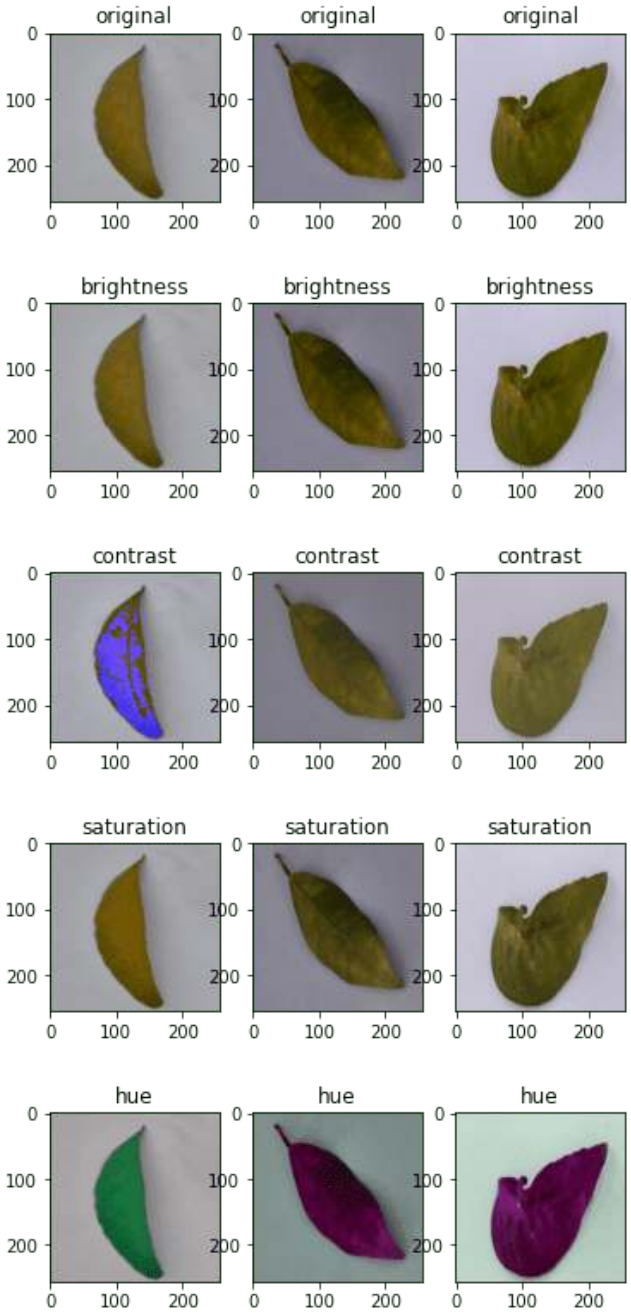


*Figure 27.8: Example transformation from Listing 27.16*

Below is the complete code to display all of the above:

```
from tensorflow.keras.utils import image_dataset_from_directory
import tensorflow as tf
import matplotlib.pyplot as plt

# use image_dataset_from_directory() to load images, with image size scaled to 256x256
PATH='.../Citrus/Leaves'  # modify to your path
ds = image_dataset_from_directory(PATH,
                                  validation_split=0.2, subset="training",
                                  image_size=(256,256), interpolation="mitchellcubic",
                                  crop_to_aspect_ratio=True,
                                  seed=42, shuffle=True, batch_size=32)

# Visualize tf.image augmentations

fig, ax = plt.subplots(5, 3, figsize=(6,14))
for images, labels in ds.take(1):
    for i in range(3):
        # original
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # resize
        h = int(256 * tf.random.uniform([], minval=0.8, maxval=1.2))
        w = int(256 * tf.random.uniform([], minval=0.8, maxval=1.2))
        ax[1][i].imshow(tf.image.resize(images[i], [h,w]).numpy().astype("uint8"))
        ax[1][i].set_title("resize")
        # crop
        y, x, h, w = (128 * tf.random.uniform((4,))).numpy().astype("uint8")
        ax[2][i].imshow(tf.image.crop_to_bounding_box(images[i], y, x, h, w)
                        .numpy().astype("uint8"))
        ax[2][i].set_title("crop")
        # central crop
        x = tf.random.uniform([], minval=0.4, maxval=1.0)
        ax[3][i].imshow(tf.image.central_crop(images[i], x).numpy().astype("uint8"))
        ax[3][i].set_title("central crop")
        # crop to (h,w) at random offset
        h, w = (256 * tf.random.uniform((2,))).numpy().astype("uint8")
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
        ax[4][i].imshow(tf.image.stateless_random_crop(images[i], [h,w,3], seed)
                        .numpy().astype("uint8"))
        ax[4][i].set_title("random crop")
plt.show()

fig, ax = plt.subplots(5, 3, figsize=(6,14))
for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # flip
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
        ax[1][i].imshow(tf.image.stateless_random_flip_left_right(images[i], seed)
                        .numpy().astype("uint8"))
        ax[1][i].set_title("flip left-right")
        # flip
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
```

```
        ax[2][i].imshow(tf.image.stateless_random_flip_up_down(images[i], seed)
                        .numpy().astype("uint8"))
        ax[2][i].set_title("flip up-down")
        # sobel edge
        sobel = tf.image.sobel_edges(images[i:i+1])
        ax[3][i].imshow(sobel[0, ..., 0].numpy().astype("uint8"))
        ax[3][i].set_title("sobel y")
        # sobel edge
        ax[4][i].imshow(sobel[0, ..., 1].numpy().astype("uint8"))
        ax[4][i].set_title("sobel x")
plt.show()

fig, ax = plt.subplots(5, 3, figsize=(6,14))
for images, labels in ds.take(1):
    for i in range(3):
        ax[0][i].imshow(images[i].numpy().astype("uint8"))
        ax[0][i].set_title("original")
        # brightness
        seed = tf.random.uniform((2,), minval=0, maxval=65536).numpy().astype("int32")
        ax[1][i].imshow(tf.image.stateless_random_brightness(images[i], 0.3, seed)
                        .numpy().astype("uint8"))
        ax[1][i].set_title("brightness")
        # contrast
        ax[2][i].imshow(tf.image.stateless_random_contrast(images[i], 0.7, 1.3, seed)
                        .numpy().astype("uint8"))
        ax[2][i].set_title("contrast")
        # saturation
        ax[3][i].imshow(tf.image.stateless_random_saturation(images[i], 0.7, 1.3, seed)
                        .numpy().astype("uint8"))
        ax[3][i].set_title("saturation")
        # hue
        ax[4][i].imshow(tf.image.stateless_random_hue(images[i], 0.3, seed)
                        .numpy().astype("uint8"))
        ax[4][i].set_title("hue")
plt.show()
```

*Listing 27.17: Various augmentations using `tf.image` API*

These augmentation functions should be enough for most uses. But if you have some specific ideas on augmentation, you would probably need a better image processing library. OpenCV and Pillow are common but powerful libraries that allow you to transform images better.

## 27.5 Using Preprocessing Layers in Neural Networks

You used the Keras preprocessing layers as functions in the examples above. But they can also be used as layers in a neural network. It is trivial to use. Below is an example of how you can incorporate a preprocessing layer into a classification network and train it using a dataset:

```
from tensorflow.keras.utils import image_dataset_from_directory
import tensorflow as tf
import matplotlib.pyplot as plt

# use image_dataset_from_directory() to load images, with image size scaled to 256x256
PATH='.../Citrus/Leaves'  # modify to your path
ds = image_dataset_from_directory(PATH,
                                  validation_split=0.2, subset="training",
                                  image_size=(256,256), interpolation="mitchellcubic",
                                  crop_to_aspect_ratio=True,
                                  seed=42, shuffle=True, batch_size=32)

AUTOTUNE = tf.data.AUTOTUNE
ds = ds.cache().prefetch(buffer_size=AUTOTUNE)

num_classes = 5
model = tf.keras.Sequential([
  tf.keras.layers.RandomFlip("horizontal_and_vertical"),
  tf.keras.layers.RandomRotation(0.2),
  tf.keras.layers.Rescaling(1/127.0, offset=-1),
  tf.keras.layers.Conv2D(32, 3, activation='relu'),
  tf.keras.layers.MaxPooling2D(),
  tf.keras.layers.Conv2D(32, 3, activation='relu'),
  tf.keras.layers.MaxPooling2D(),
  tf.keras.layers.Conv2D(32, 3, activation='relu'),
  tf.keras.layers.MaxPooling2D(),
  tf.keras.layers.Flatten(),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dense(num_classes)
])

model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

model.fit(ds, epochs=3)
```

*Listing 27.18: Using preprocessing layer as part of a neural network*

Running this code gives the following output:

```
Found 609 files belonging to 5 classes.
Using 488 files for training.
Epoch 1/3
16/16 [==============================] - 5s 253ms/step - loss: 1.4114 - accuracy: 0.4283
Epoch 2/3
16/16 [==============================] - 4s 259ms/step - loss: 0.8101 - accuracy: 0.6475
Epoch 3/3
16/16 [==============================] - 4s 267ms/step - loss: 0.7015 - accuracy: 0.7111
```

*Output 27.4: Example output on training a network with preprocessing layers*

In the code above, you created the dataset with `cache()` and `prefetch()`. This is a performance technique to allow the dataset to prepare data asynchronously while the neural

network is trained. This would be significant if the dataset has some other augmentation assigned using the `map()` function.

You will see some improvement in accuracy if you remove the `RandomFlip` and `RandomRotation` layers because you make the problem easier. However, as you want the network to predict well on a wide variation of image quality and properties, using augmentation can help you resulting network become more powerful.

## 27.6 Further Reading

Below are documentations from TensorFlow that are related to the examples above:

### APIs

*tf.data.Dataset*. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/data/Dataset
*Module: tf.image*. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/image

### Articles

*Citrus Leaves*. TensorFlow Datasets.
https://www.tensorflow.org/datasets/catalog/citrus_leaves
*Load and preprocess images*. TensorFlow Tutorials.
https://www.tensorflow.org/tutorials/load_data/images
*Data augmentation*. TensorFlow Tutorials.
https://www.tensorflow.org/tutorials/images/data_augmentation
*Better performance with the tf.data API*. TensorFlow guide.
https://www.tensorflow.org/guide/data_performance

## 27.7 Summary

In this chapter, you have seen how you can use the `tf.data` dataset with image augmentation functions from Keras and TensorFlow.

Specifically, you learned:

- How to use the preprocessing layers from Keras, both as a function and as part of a neural network
- How to create your own image augmentation function and apply it to the dataset using the `map()` function
- How to use the functions provided by the `tf.image` module for image augmentation

In the next chapter, you will see an example of doing classification using CNN and how a deeper network can improve accuracy.

# Project: Object Classification in Photographs

28

A difficult problem where traditional neural networks fall down is called object recognition. It is where a model is able to identify objects in images. In this chapter, you will discover how to develop and evaluate deep learning models for object recognition in Keras. After completing this chapter, you will know:

- ▷ About the CIFAR-10 object classification dataset and how to load and use it in Keras
- ▷ How to create a simple Convolutional Neural Network for object recognition
- ▷ How to lift performance by creating deeper Convolutional Neural Networks

Let's get started.

**Note:** You may want to speed up the computation for this chapter by using GPU rather than CPU hardware, such as the process described in Appendix C. This is a suggestion, not a requirement. The code will work just fine on the CPU.

## Overview

This chapter is divided into five sections; they are:

- ▷ The CIFAR-10 Dataset
- ▷ Loading the CIFAR-10 Dataset in Keras
- ▷ Simple Convolutional Neural Network for CIFAR-10
- ▷ Larger Convolutional Neural Network for CIFAR-10
- ▷ Extensions To Improve Model Performance

## 28.1 The CIFAR-10 Dataset

The problem of automatically classifying objects in photographs is difficult because of the nearly infinite number of permutations of objects, positions, lighting, and so on. It's a tough problem. This is a well-studied problem in computer vision and, more recently, an important demonstration of the capability of deep learning. A standard computer vision and deep

learning dataset for this problem was developed by the Canadian Institute for Advanced Research (CIFAR).

The CIFAR-10 dataset consists of 60,000 photos divided into 10 classes (hence the name CIFAR-10). Classes include common objects such as airplanes, automobiles, birds, cats, and so on. The dataset is split in a standard way, where 50,000 images are used for training a model and the remaining 10,000 for evaluating its performance. The photos are in color with red, green, and blue channels, but are small, measuring $32 \times 32$ pixel squares.

State-of-the-art results can be achieved using very large convolutional neural networks. You can learn about state-of-the-art results on CIFAR-10 on Rodrigo Benenson's webpage. Model performance is reported in classification accuracy, with very good performance above 90%, with human performance on the problem at 94% and state-of-the-art results at 96% at the time of writing.

## 28.2 Loading The CIFAR-10 Dataset in Keras

The CIFAR-10 dataset can easily be loaded in Keras. Keras has the facility to automatically download standard datasets like CIFAR-10 and store them in the `~/.keras/datasets` directory using the `cifar10.load_data()` function. This dataset is large at 163 megabytes, so it may take a few minutes to download. Once downloaded, subsequent calls to the function will load the dataset ready for use.

The dataset is stored as *pickled* training and test sets, ready for use in Keras. Each image is represented as a three-dimensional matrix, with dimensions for color (red, green, blue), width, and height. We can plot images directly using matplotlib.

```
# Plot ad hoc CIFAR10 instances
from tensorflow.keras.datasets import cifar10
import matplotlib.pyplot as plt
# load data
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
# create a grid of 3x3 images
for i in range(0, 9):
    plt.subplot(330 + 1 + i)
    plt.imshow(X_train[i])
# show the plot
plt.show()
```

*Listing 28.1: Load and plot sample CIFAR-10 images*

Running the code creates a $3 \times 3$ plot of photographs. The images have been scaled up from their small $32 \times 32$ size, but you can clearly see trucks, horses, and cars. You can also see some distortion in the images that have been forced to the square aspect ratio.

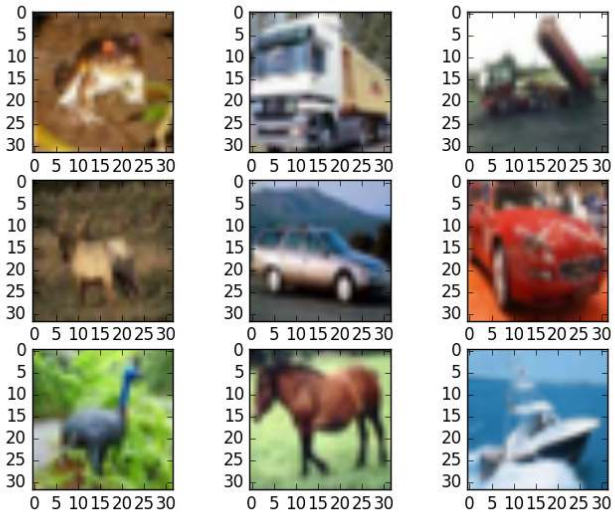

*Figure 28.1: Small sample of CIFAR-10 images*

## 28.3 Simple Convolutional Neural Network for CIFAR-10

The CIFAR-10 problem is best solved using a convolutional neural network (CNN). You can quickly start by importing all the classes and functions you will need in this example.

```
# Simple CNN model for CIFAR-10
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.utils import to_categorical
...
```

*Listing 28.2: Load classes and functions*

Next, you can load the CIFAR-10 dataset.

```
...
# load data
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
```

*Listing 28.3: Load the CIFAR-10 dataset*

The pixel values range from 0 to 255 for each of the red, green, and blue channels. It is good practice to work with normalized data. Because the input values are well understood, you can easily normalize to the range 0 to 1 by dividing each value by the maximum observation,

which is 255. Note that the data is loaded as integers, so you must cast it to floating point values in order to perform the division.

```
...
# normalize inputs from 0-255 to 0.0-1.0
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train = X_train / 255.0
X_test = X_test / 255.0
```

*Listing 28.4: Normalize the CIFAR-10 dataset*

The output variables are defined as a vector of integers from 0 to 1 for each class. You can use a one-hot encoding to transform them into a binary matrix to best model the classification problem. There are ten classes for this problem, so you can expect the binary matrix to have a width of 10.

```
...
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
```

*Listing 28.5: One-hot encode the output variable*

Let's start by defining a simple CNN structure as a baseline and evaluate how well it performs on the problem. You will use a structure with two convolutional layers followed by max pooling and a flattening out of the network to fully connected layers to make predictions. You baseline network structure can be summarized as follows:

1. Convolutional input layer, 32 feature maps with a size of $3 \times 3$, a rectifier activation function, and a weight constraint of max norm set to 3
2. Dropout set to 20%
3. Convolutional layer, 32 feature maps with a size of $3 \times 3$, a rectifier activation function, and a weight constraint of max norm set to 3
4. Max Pool layer with the size $2 \times 2$
5. Flatten layer
6. Fully connected layer with 512 units and a rectifier activation function
7. Dropout set to 50%
8. Fully connected output layer with 10 units and a softmax activation function

A logarithmic loss function is used with the stochastic gradient descent optimization algorithm configured with a large momentum and weight decay, starting with a learning rate of 0.01. A visualization of the network structure is provided below.

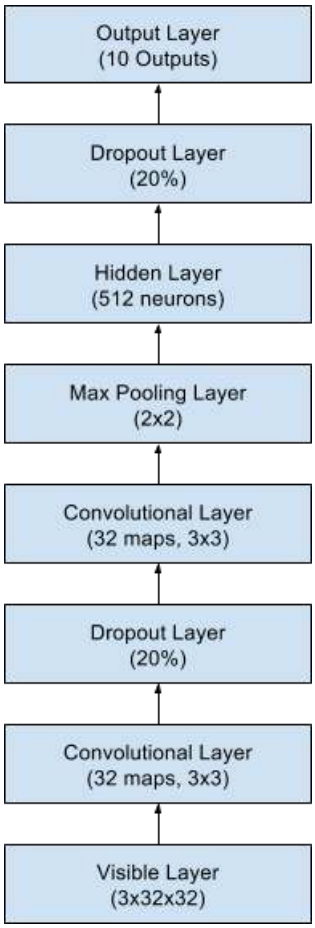


*Figure 28.2: Summary of the convolutional neural network structure*

```
...
# Create the model
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(32, 32, 3), padding='same',
                 activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.2))
model.add(Conv2D(32, (3, 3), activation='relu', padding='same',
                 kernel_constraint=MaxNorm(3)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(512, activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
# Compile model
epochs = 25
lrate = 0.01
decay = lrate/epochs
sgd = SGD(learning_rate=lrate, momentum=0.9, decay=decay, nesterov=False)
model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
model.summary()
```

*Listing 28.6: Define and compile the CNN model*

You fit this model with 25 epochs and a batch size of 32. A small number of epochs was chosen to help keep this chapter moving. Usually the number of epochs would be one or two orders of magnitude larger for this problem. Once the model is fit, you evaluate it on the test dataset and print out the classification accuracy.

```
...
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=epochs, batch_size=32)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 28.7: Evaluate the accuracy of the CNN model*

Tying this all together, the complete example is listed below.

```
# Simple CNN model for the CIFAR-10 Dataset
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.utils import to_categorical
# load data
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
# normalize inputs from 0-255 to 0.0-1.0
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train = X_train / 255.0
X_test = X_test / 255.0
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
# Create the model
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(32, 32, 3), padding='same',
          activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.2))
model.add(Conv2D(32, (3, 3), activation='relu', padding='same',
          kernel_constraint=MaxNorm(3)))
model.add(MaxPooling2D())
model.add(Flatten())
model.add(Dense(512, activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
# Compile model
epochs = 25
```

```
lrate = 0.01
decay = lrate/epochs
sgd = SGD(learning_rate=lrate, momentum=0.9, decay=decay, nesterov=False)
model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
model.summary(line_length=72)
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=epochs, batch_size=32)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 28.8: CNN model for CIFAR-10 problem*

Running this example provides the results below. First, the network structure is summarized, which confirms the design was implemented correctly.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 conv2d (Conv2D)              (None, 32, 32, 32)        896

 dropout (Dropout)            (None, 32, 32, 32)        0

 conv2d_1 (Conv2D)            (None, 32, 32, 32)        9248

 max_pooling2d (MaxPooling2D) (None, 16, 16, 32)        0

 flatten (Flatten)            (None, 8192)              0

 dense (Dense)                (None, 512)               4194816

 dropout_1 (Dropout)          (None, 512)               0

 dense_1 (Dense)              (None, 10)                5130

=================================================================
Total params: 4,210,090
Trainable params: 4,210,090
Non-trainable params: 0
_________________________________________________________________
```

*Output 28.1: Network structure of the CNN model for CIFAR-10 problem*

The classification accuracy and loss is printed each epoch on both the training and test datasets.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

The model is evaluated on the test set and achieves an accuracy of 70.5%, which is not excellent.

```
...
Epoch 20/25
1563/1563 [==============================] - 34s 22ms/step - loss: 0.3001 - accuracy: 0.8944 - val_l
Epoch 21/25
1563/1563 [==============================] - 35s 23ms/step - loss: 0.2783 - accuracy: 0.9021 - val_l
Epoch 22/25
1563/1563 [==============================] - 35s 22ms/step - loss: 0.2623 - accuracy: 0.9084 - val_l
Epoch 23/25
1563/1563 [==============================] - 33s 21ms/step - loss: 0.2536 - accuracy: 0.9104 - val_l
Epoch 24/25
1563/1563 [==============================] - 34s 22ms/step - loss: 0.2383 - accuracy: 0.9180 - val_l
Epoch 25/25
1563/1563 [==============================] - 37s 24ms/step - loss: 0.2245 - accuracy: 0.9219 - val_l
Accuracy: 70.50%
```

*Output 28.2: Sample output for the CNN model*

You can improve the accuracy significantly by creating a much deeper network. This is what you will look at in the next section.

## 28.4 Larger Convolutional Neural Network for CIFAR-10

You have seen that a simple CNN performs poorly on this complex problem. In this section, you will look at scaling up the size and complexity of your model. Let's design a deep version of the simple CNN above. You can introduce an additional round of convolutions with many more feature maps. You will use the same pattern of convolutional, dropout, convolutional and max pooling layers.

This pattern will be repeated three times with 32, 64, and 128 feature maps. The effect is an increasing number of feature maps with a smaller and smaller size given the max pooling layers. Finally, an additional and larger `Dense` layer will be used at the output end of the network in an attempt to better translate the large number of feature maps to class values. A summary of the new network architecture is as follows:

1. Convolutional input layer, 32 feature maps with a size of $3 \times 3$ and a rectifier activation function.
2. Dropout layer at 20%.
3. Convolutional layer, 32 feature maps with a size of $3 \times 3$ and a rectifier activation function.
4. Max Pool layer with size $2 \times 2$.
5. Convolutional layer, 64 feature maps with a size of $3 \times 3$ and a rectifier activation function.
6. Dropout layer at 20%.
7. Convolutional layer, 64 feature maps with a size of $3 \times 3$ and a rectifier activation function.
8. Max Pool layer with size $2 \times 2$.
9. Convolutional layer, 128 feature maps with a size of $3 \times 3$ and a rectifier activation function.

10. Dropout layer at 20%.
11. Convolutional layer, 128 feature maps with a size of $3 \times 3$ and a rectifier activation function.
12. Max Pool layer with size $2 \times 2$.
13. Flatten layer.
14. Dropout layer at 20%.
15. Fully connected layer with 1,024 units and a rectifier activation function.
16. Dropout layer at 20%.
17. Fully connected layer with 512 units and a rectifier activation function.
18. Dropout layer at 20%.
19. Fully connected output layer with 10 units and a softmax activation function.

You can very easily define this network topology in Keras as follows:

```
...
# Create the model
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(32, 32, 3), activation='relu', padding='same'))
model.add(Dropout(0.2))
model.add(Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D())
model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(Dropout(0.2))
model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D())
model.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(Dropout(0.2))
model.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D())
model.add(Flatten())
model.add(Dropout(0.2))
model.add(Dense(1024, activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.2))
model.add(Dense(512, activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
# Compile model
epochs = 25
lrate = 0.01
decay = lrate/epochs
sgd = SGD(learning_rate=lrate, momentum=0.9, decay=decay, nesterov=False)
model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
model.summary()
...
```

*Listing 28.9: Defining a larger CNN for CIFAR-10*

You can fit and evaluate this model using the same procedure from above and the same number of epochs but a larger batch size of 64, found through some minor experimentation.

```
...
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=epochs, batch_size=64)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 28.10: Fitting the larger CNN with a larger batch size*

Tying this all together, the complete example is listed below.

```
# Large CNN model for the CIFAR-10 Dataset
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.utils import to_categorical
# load data
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
# normalize inputs from 0-255 to 0.0-1.0
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train = X_train / 255.0
X_test = X_test / 255.0
# one-hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
# Create the model
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(32, 32, 3), activation='relu', padding='same'))
model.add(Dropout(0.2))
model.add(Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D())
model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(Dropout(0.2))
model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D())
model.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(Dropout(0.2))
model.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D())
model.add(Flatten())
model.add(Dropout(0.2))
model.add(Dense(1024, activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.2))
model.add(Dense(512, activation='relu', kernel_constraint=MaxNorm(3)))
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
```

```
# Compile model
epochs = 25
lrate = 0.01
decay = lrate/epochs
sgd = SGD(learning_rate=lrate, momentum=0.9, decay=decay, nesterov=False)
model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
model.summary()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=epochs, batch_size=64)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 28.11: Large CNN model for CIFAR-10 problem*

Running this example prints the classification accuracy and loss on the training and test datasets for each epoch.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

The estimate of classification accuracy for the final model is 79.5% which is nine points better than our simpler model.

```
...
Epoch 20/25
782/782 [==============================] - 50s 64ms/step - loss: 0.4949 - accuracy: 0.8237 - val_los
Epoch 21/25
782/782 [==============================] - 51s 65ms/step - loss: 0.4794 - accuracy: 0.8308 - val_los
Epoch 22/25
782/782 [==============================] - 50s 64ms/step - loss: 0.4660 - accuracy: 0.8347 - val_los
Epoch 23/25
782/782 [==============================] - 50s 64ms/step - loss: 0.4523 - accuracy: 0.8395 - val_los
Epoch 24/25
782/782 [==============================] - 50s 64ms/step - loss: 0.4344 - accuracy: 0.8454 - val_los
Epoch 25/25
782/782 [==============================] - 50s 64ms/step - loss: 0.4231 - accuracy: 0.8487 - val_los
Accuracy: 79.50%
```

*Output 28.3: Sample output for fitting the larger CNN model*

## 28.5 Extensions to Improve Model Performance

You have achieved good results on this very difficult problem, but you are still a good way from achieving world-class results. Below are some ideas that you can try to extend upon the model and improve model performance.

- ▷ **Train for More Epochs**. Each model was trained for a very small number of epochs, 25. It is common to train large convolutional neural networks for hundreds or thousands of

epochs. You should expect performance gains can be achieved by significantly raising the number of training epochs.

- ▷ **Image Data Augmentation**. The objects in the image vary in their position. Another boost in model performance can likely be achieved by using some data augmentation. Methods such as standardization, random shifts, and horizontal image flips may be beneficial.
- ▷ **Deeper Network Topology**. The larger network presented is deep, but larger networks could be designed for the problem. This may involve more feature maps closer to the input and perhaps less aggressive pooling. Additionally, standard convolutional network topologies that have been shown useful may be adopted and evaluated on the problem.

## 28.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

### Articles

Alex Krizhevsky. *The CIFAR-10 dataset.*
https://www.cs.toronto.edu/~kriz/cifar.html

Rodrigo Benenson. *What is the class of this image?* Classification datasets results, 2016.
https://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html

## 28.7 Summary

In this chapter, you discovered how to create deep learning models in Keras for object recognition in photographs. After working through this chapter, you learned:

- ▷ About the CIFAR-10 dataset and how to load it in Keras and plot ad hoc examples from the dataset
- ▷ How to train and evaluate a simple Convolutional Neural Network on the problem
- ▷ How to expand a simple convolutional neural network into a deep convolutional neural network in order to boost performance on the difficult problem
- ▷ How to use data augmentation to get a further boost on the difficult object recognition problem

In the next chapter, you will see that CNN can also be applied to non-image data.

# Project: Predict Sentiment from Movie Reviews

29

Sentiment analysis is a natural language processing problem where text is understood, and the underlying intent is predicted. In this chapter, you will discover how you can predict the sentiment of movie reviews as either positive or negative in Python using the Keras deep learning library. After completing this chapter, you will know:

- ▷ About the IMDB sentiment analysis problem for natural language processing and how to load it in Keras
- ▷ How to use word embedding in Keras for natural language problems
- ▷ How to develop and evaluate a multilayer perceptron model for the IMDB problem
- ▷ How to develop a one-dimensional convolutional neural network model for the IMDB problem

Let's get started.

## Overview

This chapter is divided into five sections; they are:

- ▷ IMDB Movie Review Sentiment Problem
- ▷ Load the IMDB Dataset With Keras
- ▷ Word Embeddings
- ▷ Simple Multilayer Perceptron Model for the IMDB Dataset
- ▷ One-Dimensional Convolutional Neural Network Model for the IMDB Dataset

## 29.1 IMDB Movie Review Sentiment Problem

The dataset is the *Large Movie Review Dataset*, often referred to as the IMDB dataset. The IMDB dataset contains 25,000 highly polar movie reviews (good or bad) for training and the same amount again for testing. The problem is to determine whether a given movie review has a positive or negative sentiment.

The data was collected by Stanford researchers and used in Maas et al., "Learning Word Vectors for Sentiment Analysis" where a split of 50-50 of the data was used for training and test. An accuracy of 88.89% was achieved.

## 29.2 Load the IMDB Dataset with Keras

Keras provides built-in access to the IMDB dataset. The `tf.keras.datasets.imdb.load_data()` allows you to load the dataset in a format that is ready for use in neural networks and deep learning models. The words have been replaced by integers that indicate the absolute popularity of the word in the dataset. The sentences in each review are therefore comprised of a sequence of integers.

Calling `imdb.load_data()` the first time will download the IMDB dataset to your computer and store it in your home directory under `~/.keras/datasets/imdb.npz` as a 17MB file. Usefully, the `imdb.load_data()` provides additional arguments, including the number of top words to load (where words with a lower integer are marked as zero in the returned data), the number of top words to skip (to avoid the repeated use of *the*), and the maximum length of reviews to support. Let's load the dataset and calculate some properties of it. You will start by loading some libraries and the entire IMDB dataset as a training dataset.

```
import numpy as np
from tensorflow.keras.datasets import imdb
import matplotlib.pyplot as plt
# load the dataset
(X_train, y_train), (X_test, y_test) = imdb.load_data()
X = np.concatenate((X_train, X_test), axis=0)
y = np.concatenate((y_train, y_test), axis=0)
...
```

*Listing 29.1: Load the IMDB dataset*

Next, you can display the shape of the training dataset.

```
...
# summarize size
print("Training data: ")
print(X.shape)
print(y.shape)
```

*Listing 29.2: Display the shape of the IMDB dataset*

Running this snippet, you can see that there are 50,000 records.

```
Training data:
(50000,)
(50000,)
```

*Output 29.1: Output of the shape of the IMDB dataset*

You can also print the unique class values.

```
...
# Summarize number of classes
print("Classes: ")
print(np.unique(y))
```

*Listing 29.3: Display the classes in IMDB dataset*

You can see it is a binary classification problem for good and bad sentiment in the review.

```
Classes:
[0 1]
```

*Output 29.2: Output of the classes of the IMDB dataset*

Next, you can get an idea of the total number of unique words in the dataset.

```
...
# Summarize number of words
print("Number of words: ")
print(len(np.unique(np.hstack(X))))
```

*Listing 29.4: Display the number of unique words in the IMDB dataset*

Interestingly, you can see that there are just under 100,000 words across the entire dataset.

```
Number of words:
88585
```

*Output 29.3: Output of the classes of unique words in the IMDB dataset*

Finally, you can get an idea of the average review length.

```
...
# Summarize review length
print("Review length: ")
result = [len(x) for x in X]
print("Mean %.2f words (%f)" % (np.mean(result), np.std(result)))
# plot review length
plt.subplot(121)
plt.boxplot(result)
plt.subplot(122)
plt.hist(result)
plt.show()
```

*Listing 29.5: Plot the distriobution of review lengths*

You can see that the average review has just under 300 words with a standard deviation of just over 200 words.

```
Review length:
Mean 234.76 words (172.911495)
```

*Output 29.4: Output of the distribution summary for review length*

Looking at the box and whisker plot and the histogram for the review lengths in words, you can see an exponential distribution that you can probably cover the mass of the distribution with a clipped length of 400 to 500 words.

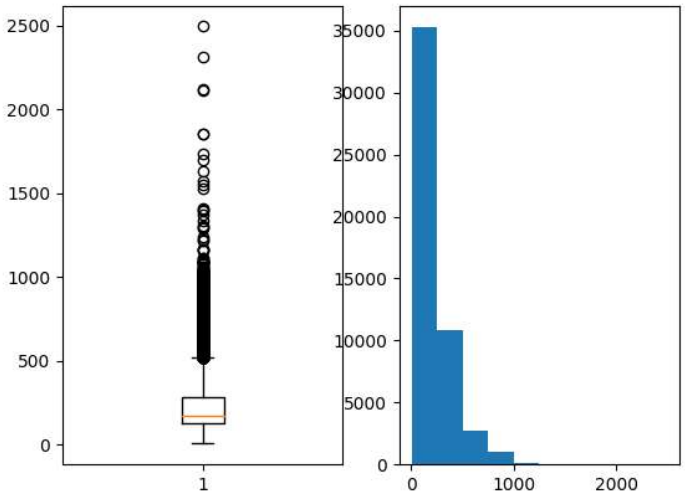


*Figure 29.1: Review length in words for IMDB dataset*

## 29.3 Word Embeddings

A recent breakthrough in the field of natural language processing is called word embedding. This technique is where words are encoded as real-valued vectors in a high-dimensional space, where the similarity between words in terms of meaning translates to closeness in the vector space. Discrete words are mapped to vectors of continuous numbers. This is useful when working with natural language problems with neural networks and deep learning models as they require numbers as input values.

Keras provides a convenient way to convert positive integer representations of words into a word embedding by an `Embedding` layer. The layer takes arguments that define the mapping, including the maximum number of expected words, also called the vocabulary size (e.g., the largest integer value that will be seen as an input). The layer also allows you to specify the dimensionality for each word vector, called the output dimension.

You want to use a word embedding representation for the IMDB dataset. Let's say that you are only interested in the first 5,000 most used words in the dataset. Therefore, your vocabulary size will be 5,000. You can choose to use a 32-dimensional vector to represent each word. Finally, you may choose to cap the maximum review length at 500 words, truncating reviews longer than that and padding reviews shorter than that with 0 values. You will load the IMDB dataset as follows:

```
...
imdb.load_data(num_words=5000)
```

*Listing 29.6: Load only the top 5000 words in the IMDB review*

You will then use the Keras utility to truncate or pad the dataset to a length of 500 for each observation using the `sequence.pad_sequences()` function.

```
...
X_train = sequence.pad_sequences(X_train, maxlen=500)
X_test = sequence.pad_sequences(X_test, maxlen=500)
```

*Listing 29.7: Pad reviews in the IMDB dataset*

Finally, later on, the first layer of your model would be a word embedding layer created using the `Embedding` class as follows:

```
...
Embedding(5000, 32, input_length=500)
```

*Listing 29.8: Define a word embedding representation*

The output of this first layer would be a matrix with the size $32 \times 500$ for a given movie review training or test pattern in integer format. Now that you know how to load the IMDB dataset in Keras and how to use a word embedding representation for it, let's develop and evaluate some models.

## 29.4 Simple Multilayer Perceptron Model for the IMDB Dataset

You can start by developing a simple multilayer perceptron model with a single hidden layer. The word embedding representation is a true innovation, and you will demonstrate what would have been considered world-class results in 2011 with a relatively simple neural network. Let's start by importing the classes and functions required for this model and initializing the random number generator to a constant value to ensure you can easily reproduce the results.

```
# MLP for the IMDB problem
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence

...
```

*Listing 29.9: Load classes and functions*

Next, you will load the IMDB dataset. You will simplify the dataset as discussed during the section on word embeddings — only the top 5,000 words will be loaded. You will also use a 50/50 split of the dataset into training and test sets. This is a good standard split methodology.

```
...
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
```

*Listing 29.10: Load and split the IMDB dataset*

You will bound reviews at 500 words, truncating longer reviews and zero-padding shorter ones.

```
...
max_words = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_words)
X_test = sequence.pad_sequences(X_test, maxlen=max_words)
```

*Listing 29.11: Pad IMDB reviews to a fixed length*

Now, you can create your model. You will use an `Embedding` layer as the input layer, setting the vocabulary to 5,000, the word vector size to 32 dimensions, and the `input_length` to 500. The output of this first layer will be a $32 \times 500$-sized matrix, as discussed in the previous section. You will flatten the `Embedded` layers' output to one dimension, then use one dense hidden layer of 250 units with a rectifier activation function. The output layer has one neuron and will use a sigmoid activation to output values of 0 and 1 as predictions. The model uses logarithmic loss and is optimized using the efficient ADAM optimization procedure.

```
...
# create the model
model = Sequential()
model.add(Embedding(top_words, 32, input_length=max_words))
model.add(Flatten())
model.add(Dense(250, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

*Listing 29.12: Define a multilayer perceptron model*

You can fit the model and use the test set as validation while training. This model overfits very quickly, so you will use very few training epochs, in this case, just 2. There is a lot of data, so you will use a batch size of 128. After the model is trained, you will evaluate its accuracy on the test dataset.

```
...
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=2, batch_size=128, verbose=1)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 29.13: Fit and evaluate the multilayer pereceptron model*

Tying all of this together, the complete code listing is provided below.

```
# MLP for the IMDB problem
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
max_words = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_words)
X_test = sequence.pad_sequences(X_test, maxlen=max_words)
# create the model
model = Sequential()
model.add(Embedding(top_words, 32, input_length=max_words))
model.add(Flatten())
model.add(Dense(250, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=2, batch_size=128, verbose=1)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 29.14: Multilayer perceptron model for the IMDB dataset*

Running this example fits the model and summarizes the estimated performance.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

You can see that this very simple model achieves a score of 87%, which is in the neighborhood of the original paper, with very minimal effort.

```
Epoch 1/2
196/196 - 4s - loss: 0.5579 - accuracy: 0.6664 - val_loss: 0.3060 - val_accuracy: 0.8700 - 4s/epoch
Epoch 2/2
196/196 - 4s - loss: 0.2108 - accuracy: 0.9165 - val_loss: 0.3006 - val_accuracy: 0.8731 - 4s/epoch
Accuracy: 87.31%
```

*Output 29.5: Output from evaluating the multilayer perceptron model*

You can likely do better if you trained this network, perhaps using a larger embedding and adding more hidden layers.

Let's try a different network type.

## 29.5 One-Dimensional Convolutional Neural Network Model for the IMDB Dataset

Convolutional neural networks were designed to honor the spatial structure in image data while being robust to the position and orientation of learned objects in the scene. This same principle can be used on sequences, such as the one-dimensional sequence of words in a movie review. The same properties that make the CNN model attractive for learning to recognize objects in images can help to learn structure in paragraphs of words, namely the techniques invariance to the specific position of features.

Keras supports one-dimensional convolutions and pooling by the `Conv1D` and `MaxPooling1D` classes, respectively. Again, let's import the classes and functions needed for this example and initialize your random number generator to a constant value so that you can easily reproduce the results.

```
# CNN for the IMDB problem
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Conv1D
from tensorflow.keras.layers import MaxPooling1D
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
```

*Listing 29.15: Import classes and functions*

You can also load and prepare the IMDB dataset as you did before.

```
...
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
# pad dataset to a maximum review length in words
max_words = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_words)
X_test = sequence.pad_sequences(X_test, maxlen=max_words)
```

*Listing 29.16: Load, split, and pad IMDB dataset*

You can now define your convolutional neural network model. This time, after the `Embedding` input layer, insert a `Conv1D` layer. This convolutional layer has 32 feature maps and reads embedded word representations' three vector elements of the word embedding at a time. The convolutional layer is followed by a `MaxPooling1D` layer with a length and stride of 2 that halves the size of the feature maps from the convolutional layer. The rest of the network is the same as the neural network above.

```
...
# create the model
model = Sequential()
```

```
model.add(Embedding(top_words, 32, input_length=max_words))
model.add(Conv1D(32, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(250, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

*Listing 29.17: Define the CNN model*

You will also fit the network the same as before.

```
...
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=2, batch_size=128, verbose=2)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 29.18: Fit and evaluate the CNN model*

Tying all of this together, the complete code listing is provided below.

```
# CNN for the IMDB problem
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Conv1D
from tensorflow.keras.layers import MaxPooling1D
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing import sequence
# load the dataset but only keep the top n words, zero the rest
top_words = 5000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
# pad dataset to a maximum review length in words
max_words = 500
X_train = sequence.pad_sequences(X_train, maxlen=max_words)
X_test = sequence.pad_sequences(X_test, maxlen=max_words)
# create the model
model = Sequential()
model.add(Embedding(top_words, 32, input_length=max_words))
model.add(Conv1D(32, 3, padding='same', activation='relu'))
model.add(MaxPooling1D())
model.add(Flatten())
model.add(Dense(250, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
          epochs=2, batch_size=128, verbose=2)
```

```
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy: %.2f%%" % (scores[1]*100))
```

*Listing 29.19: CNN model for the IMDB dataset*

Running the example, you are first presented with a summary of the network structure (not shown here). You can see your convolutional layer preserves the dimensionality of your `Embedding` input layer of 32-dimensional input with a maximum of 500 words. The pooling layer compresses this representation by halving it.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running the example offers a slight but welcome improvement over the neural network model above with an accuracy of 87%.

```
Epoch 1/2
196/196 - 5s - loss: 0.4661 - accuracy: 0.7467 - val_loss: 0.2763 - val_accuracy: 0.8860 - 5s/epoch
Epoch 2/2
196/196 - 5s - loss: 0.2195 - accuracy: 0.9144 - val_loss: 0.3063 - val_accuracy: 0.8764 - 5s/epoch
Accuracy: 87.64%
```

*Output 29.6: Output from evaluating the CNN model*

Again, there is a lot of opportunity for further optimization, such as using deeper and/or larger convolutional layers. One interesting idea is to set the max pooling layer to use an input length of 500. This would compress each feature map to a single 32-length vector and may boost performance.

## 29.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

### Papers

Andrew L. Maas et al. "Learning Word Vectors for Sentiment Analysis". In: *The 49th Annual Meeting of the Association for Computational Linguistics*. 2011.
http://ai.stanford.edu/~amaas/papers/wvSent_acl2011.pdf

### APIs

*IMDB movie review sentiment classification dataset*. Keras API reference.
https://keras.io/api/datasets/imdb/
*Embedding Layer*. Keras API reference.
https://keras.io/api/layers/core_layers/embedding/

### Articles

*Large Movie Review Dataset.*
http://ai.stanford.edu/~amaas/data/sentiment/
*word embedding.* Wikipedia.
https://en.wikipedia.org/wiki/Word_embedding

## 29.7 Summary

In this chapter, you discovered the IMDB sentiment analysis dataset for natural language processing. You learned how to develop deep learning models for sentiment analysis, including:

- ▷ How to load and review the IMDB dataset within Keras
- ▷ How to develop a large neural network model for sentiment analysis
- ▷ How to develop a one-dimensional convolutional neural network model for sentiment analysis

This is the end of this part. In the next part, you will see a different kind of neural network.